index

October 13, 2026

Programmering

VIKTIGT! Instruktioner till hur du använder Jupyters Notebook (klicka på länken)

Angående kapitelindelningen:

Nivå 1 innehåller smakbitar från några olika delar av python. Inte så systematiskt men saker som man kan behöva för att komma igång med att skriva program.

Nivå 2 innehåller fördjupningar av kommandon och begrepp i nivå 1.

Nivå 3 innehåller en mer systematisk översikt över vissa kommandon och begrepp. Men också en fördjupad användning i samband med olika matematiska begrepp. Ej klar.

Rubriker över nivå 3 är mer ordnat efter matematiska teman än programmering i sig.

Nivå 1 och 2 utgör en grundläggande kurs.

0.1 1 Grundläggande, Nivå 1

0.1.1 1.1 Introduktion antal uppgifter (4)

0.1.2 1.2 Variabeltilldelning: x= (2)

0.1.3 1.3 Att göra val: if...else (6)

0.1.4 1.4 input() (1)

0.1.5 1.5 Ett Enkelt Program

0.1.6 1.6 Importera färdiga Moduler (1)

0.1.7 1.7 Loopar och lite om listor (6)

0.1.8 1.8 Komplexa tal (2)

0.2 2 Nivå 2

0.2.1 2.1 Listor (7)

0.2.2 2.2 Strängar (2)

0.2.3 2.3 Programexempel

0.2.4 2.4 Funktioner (4)

0.2.5 2.5 Plotta (8)

0.2.6 2.6 Skriva och läsa filer (2)

0.2.7 2.7 Egna Moduler (2)

0.2.8 2.8 Print med formatering (2)

0.2.9 2.9 Variablers räckvidd (1)

0.3 3 Nivå 3

0.3.1 3.1 Kommandon ej klar

0.3.2 3.2 Generiska problem: sökning, sortering

0.3.3 3.3 Plot 3D

0.3.4 3.4 Funktionsanpassningar 1

0.3.5 3.5 Statistik 1 ej klart

0.3.6 3.6 Exempel

0.3.7 3.7 Fler datastrukturer ej klar

0.3.8 3.8 Hur plottning fungerar ej klar

0.3.9 3.9 Lösningar till polynomekvationer ej klar

0.4 4 Linjär algebra

(inte en kurs i linjär algebra)

0.4.1 4.1 array() för vektorer och matriser (6)

0.4.2 4.2 Rita linjer och plan och vektorer

0.5 5 Flerdimensionell analys

(inte en kurs i flerdimensionell analys)

0.5.1 5.1 Parametriserade kurvor och ytor

0.5.2 5.2 Vektorfält

0.6 6 Symbolic och Scientific Python: sympy o scipy

0.6.1 6.1 Intro, Symboler och variabler

0.6.2

0.6.3 6.3 Egenvärden

0.6.4 6.4 Allmän kurvanpassning

0.7 7 Fourieranalys

0.7.1 7.1 Beräkning av Fourierserie (3)

0.7.2 7.2 Beräkning av Fouriertransform (2)

0.8 8 Större uppgifter

0.8.1 8.1 Största gemensamma delare

0.8.2 8.2 Nollställen genom intervallhalvering

0.8.3 8.3 Urvalssortering

Tidskomplexitet

Algoritm för Urvalssortering

0.8.4 8.4 Eulers metod

0.9 9 Fysik

0.9.1 9.1 Komplexa tal

0.9.2 9.2 array för vektorer och matriser

0.9.3 9.3 Eulers metod

0.9.4 9.4 Andra ordningens differentialekvation: Harmonisk oscillator

0.10 10 Grafiska gränssnitt, GUI

0.10.1 (10.1 Widgets i form av ipywidgets(kontroller: glidare, rullgardiner etc.))

0.10.2 10.2 Widgets från PySimpleGUI

0.11 11 Pandas (databehandling av större mängder)

0.11.1 11.1 Introduktion

0.11.2 Kladd

((## 8 Tkinter))

((### .1 Tkinter))

0.11.3 .1 guitester

Listor2

October 14, 2026

1 Mer om listor

Vi hade lite om listor inför loopar. Vi fördjupar oss nu i listor.

Listor är en sammansatt datatyp. Det finns fler sammansatta datatyper, t.ex.:

- mängder (eng. set) $\{1, 3, 2\}$.
- tiplar eller tuplar (eng. tuples) $(4, 2, 1)$.

Vi behandlar dem inte här men tiplar dyker upp då vi anropar olika funktioner (se 2.4) i python. Skillnaden mellan tiplar och listor är att tiplar inte kan ändras, de är så kallade 'immutables'.

1.1 Ett element i lista

Python använder hakparenteser för att ange ett element i en lista `L[0]` är första elementet, `L[2]` är tredje elementet. I listan `L=[5,3,8,9]` är L[0] talet 5 och L[2] är talet 8.

Vi kan skriva:

[2]:
```
L = [5,3,8,9]
a = L[0]
print(a)

print(L[2])
print(L[-1])
```

```
5
8
9
```

Om vi vill ändra, listor är 'mutable', ett element med ett visst index skriver vi:

[10]:
```
L = [5,3,8,9]

L[1] = 7  # Talet 3 har index 1
print(L)
```

```
[5, 7, 8, 9]
```

1.2 Del av lista

När vi anger att index ska gå mellan 2 värden i matematiken kan man ange det som L_i då $i = 2...5$ som innebär L_2, L_3, L_4 och L_5.

I python skriver man med kolon inom hakparenteserna (inte semikolon, som skrivs ;)

[1]:
```
L = [1, 2, 2, 5, 3, 7]
print(L[0:2])
```

```
[1, 2]
```

Kör koden! Men det blev bara 2 tal fastän vi skrivit 0 och 2 som gränser.

Python tolkar det som *från och med* 0 *till* 2. Dvs. inklusive 0 men exklusive 2. Argument för detta finns i dokumentationen av python. Man kan se det som L[inkl:exkl]

Denna typ av 'utsnitt' kallas på engelska för 'slice'. Observera att det är en lista, inte bara 2 tal. L[0] är däremot ett tal.

Ok, det kan man acceptera. Men vad händer då om man skriver L[0:0] där den ena nollan är inklusive och den andra exklusive? Testa är en ok metod.

[2]:
```
L = [1, 2, 2, 5, 3, 7]

print(L[0:0])
```

```
[]
```

Python svarar [] vilket betyder den tomma listan (precis som tomma mängden), inga tal från listan är med, således exklusive vinner.

Observera, igen, skillnaden mellan L[1] som är ett element och L[1:2] som är en lista! som innehåller samma element som L[1].

[1]:
```
L = [1, 2, 2, 5, 3, 7]
print(L[1])  # Element
print(L[1:2])  # Lista, slice
```

```
2
[2]
```

En lista behöver inte ha samma typer av element utan de kan blandas. En lista kan innehålla heltal, booleska variabler, strängar osv.

[3]:
```
M = [2, 3.11, -1, True, 'w']
print(M[2:5])
```

```
[-1, True, 'w']
```

Om man ska ha en lista från ett visst element till slutet räcker det med att ange starten tex. M[2:]

[13]:
```
M = [2, 3.11, -1, True, 'w']

print(M[2:])
```

```
[-1, True, 'w']
```

Och om vi vänder på det M[:2]? Då är det upp till men exklusivt index 2. M[0:2] är samma som M[:2]

[4]:
```
M = [2, 3.11, -1, True, 'w']

print(M[:2])
print(M[0:2])
```

```
[2, 3.11]
[2, 3.11]
```

Vill man ha reda på hur många element en lista innehåller så använder man len(M), som i length.

[15]:
```
M = [2, 3.11, -1, True, 'w']

print(len(M))
```

```
5
```

Om längden är 5 så går index från 0 till 4, inte 1 till 5.

Observera att listor kan adresseras direkt, de måste inte namnges. Se rad 4.

[16]:
```
L = [5, 3, 1]
print(L[1])

print([5, 3, 1][1])  # Direkt adressering
```

```
3
3
```

1.3 Operationer på listor

Vi tar bort elementet med ett visst index i en lista.

[1]:
```
L = [3,4,5]
print(L)
L[1:2] = []  # dvs element med index 1, som är talet 4
print(L)
```

```
[3, 4, 5]
[3, 5]
```

Vi kan ta bort det sista.

[8]:
```
L = [3,4,5]

L[-1:] = []  # dvs element med index -1, som är talet 5
print(L)
```

```
[3, 4]
```

Vi lägger till element från en lista sist.

[2]:
```
L = [3,4,5]
L[3:] = [1,1,0]
print(L)
```

```
[3, 4, 5, 1, 1, 0]
```

[ ]:
```
Vi lägger till element från en lista först.
```

[9]:
```
L = [3,4,5]
L[:0] = ["a","b","c"]
print(L)
```

```
['a', 'b', 'c', 3, 4, 5]
```

Man börjar undra? Kan man stoppa in elemet från en lista in i listan? Vi provar med följande:

[4]:
```
L = [3,4,5]
L[1:2] = ["a","b","c"]
print(L)
```

```
[3, 'a', 'b', 'c', 5]
```

Tydligen ersatte vi talet 4 med vår lista över strängar. Kanske kan vi få in våra element utan att ta bort något? Vi försöker med

[5]:
```
L = [3,4,5]
L[1:1] = ["a","b","c"]
print(L)
```

```
[3, 'a', 'b', 'c', 4, 5]
```

1.3.1 Ett annat sätt att lägga till ett element sist

Att lägga till ett element *sist* i en lista kan göras med något som kallas för metod enligt mallen `objekt.metod`. Objektet är variabeln och metod är vad som ska utföras på variabeln.

Vi använder metoden `append()` på ett objekt som är en lista.

[8]:
```
N = [4, 3, 5, 3, -1, 0]
print(N)

N.append(12)  # objekt.metod()
print(N)
```

```
[4, 3, 5, 3, -1, 0]
[4, 3, 5, 3, -1, 0, 12]
```

Observera att `N = N + [12]` gör samma sak.

Observera också att kommandot är inte `N = N.append(12)`. Man behöver ingen tilldelning. Det kallas att man gör ändringen 'in place', dvs på plats direkt i objektet.

1.3.2 Ta bort element

Tidigare tog vi bort element med ett visst index, oavsett elementets värde.

Med metoder kan vi ta bort ett visst värde oavsett vilket index det har.

Och vi kan ta bort ett värde tex. en av de två 3:orna. `remove(3)` tar bort den 3:a med lägst index.

[17]:
```
N = [4, 3, 5]

print(N)
N.remove(3)
print(N)
```

```
[4, 3, 5]
[4, 5]
```

Om det fler än en 3:a tas den första bort.

[18]:
```
N = [4, 3, 5, 3]

print(N)
N.remove(3)
print(N)
```

```
[4, 3, 5, 3]
[4, 5, 3]
```

`.remove()` kan inte ersättas med `N = N - [3]` (du får ett felmeddelande om du försöker). Dock kan 3, i vårt exempel, tas bort genom `N[1:2] = []`. Men det är inte samma. I ena fallet (remove) tar man bort ett tal oavsett var det är och i det andra tar man bort det tal som har ett visst index oavsett talets värde.

1.3.3 Konkatenera, antal element, reversera mm.

Konkatenera listor görs med plustecknet som vi ursprungligen använde för addition. (viktigt skilja på tecknet och operationen)

[10]:
```
S = [1, 2, 3, 4]
T = [-1, -2, -3, -3]
print(S + T)
print(T + S)
print(S + [7, 8])
print([7, 8] + S)
```

```
[1, 2, 3, 4, -1, -2, -3, -3]
[-1, -2, -3, -3, 1, 2, 3, 4]
[1, 2, 3, 4, 7, 8]
[7, 8, 1, 2, 3, 4]
```

Man kan få python till att addera elementvis men då gör man om listan till något som heter array(). En array fungerar som en vektor eller matris i matematiken. Detta finns senare i kursen.

Antalet element med ett visst värde i en lista kan räknas.

[11]:
```
L = [2,3,2,4,2]
L_nr2 = L.count(2)  # Räknar antalet 2:or i listan
print(L_nr2)
```

```
3
```

Ändra listans ordning.

[12]:
```
L = [1,2,37]
L.reverse()
print(L)
```

```
[37, 2, 1]
```

Vissa genvägar har införts för att göra det enklare. Vill man tex. starta ett program med en lista med nollor så skriver man

[13]:
```
K = 5*[0]
print(K)
```

```
[0, 0, 0, 0, 0]
```

Samma tankegång gäller för strängar

[14]:
```
W = 5*"a"
print(W)
```

```
aaaaa
```

Nu följer 2 uppgifter men det kommer mer efter uppgifterna.

1.4 Uppgifter

Uppgift 1

Skriv in listan `La=[2,9,11,18,5,4,15,7,5]` i ett program. a) Skriv ut element nr 3 (som har index 2!). b) Lägg till talet 21 sist i listan. c) Ta bort talet 5 i listan (det med lägst index).

Lösningsförslag

Uppgift 2

Vad skrivs ut av följande kod?

```
L = [2, 3, False, 'gg', [4,4]]
print(L)
print(L[2:4])
print(L[2:])
print(L[-2])

L.append(-1)
print(L)
```

```
L[2:3] = [True]  # Obs. L[2:3] = True fungerar inte!
print(L)

M=[2,3,4,5,3,1]
M.remove(3)  # Vilken 3:a tas bort?
print(M)
```

Lösningsförslag

1.5 range() i listor

I avsnittet om loopar hade vi kommandot range(). Kommandot kan också användas för att skapa listor. Vi skapar en lista med hjälp av `range()` och komandot `list()` för att omvandla till lista.

[8]:
```
L = list(range(8)) # bara stopp behöver vara med
print(L)

M = list(range(3,9,2))
print(M)
```

```
[0, 1, 2, 3, 4, 5, 6, 7]
[3, 5, 7]
```

Vi behöver kommandot `list()` som gör om `range()` till en lista.

1.6 Omvandlingar

Föregående exempel innehåller en omvandling av en inbyggd funktion (range) till en lista. Omvandlingar går att göra med fler datatyper. En populär som vi träffat på tidigare vid `input()` är int().

[16]:
```
a = "6"  # Som är en sträng
a = int(a)  # Omvandlar sträng till datatypen heltal
print(3*a)  # Nu kan vi räkna med den
```

```
18
```

Omvandlaren float() klarar även knepigare saker som att tolka decimalpunkt

[17]:
```
a = "6.2"  # Som är en sträng
a = float(a)  # Omvandlar sträng till datatypen flyttal
print(3*a)  # Nu kan vi räkna med den
```

```
18.6
```

Man börjar undra… Vad händer om vi gör

[18]:
```
a = True; b = False
a = int(a); b = int(b)
print(a, b)
```

```
1 0
```

Eller till boolean (testa själv genom att ändra b).

[19]:
```
a = 0; b = 2  # Observera att det inte är 1
a = bool(a); b = bool(b)
print(a, b)
```

```
False True
```

Komandot list() kan göra ännu mer. Strängar kan plockas i delar till en lista.

[20]:
```
a = "Det var länge se'n"
b = list(a)
print(b)
```

```
['D', 'e', 't', ' ', 'v', 'a', 'r', ' ', 'l', 'ä', 'n', 'g', 'e', ' ', 's', 'e',
"'", 'n']
```

Eller om man vill ta bort något felstavat

[21]:
```
a = "Konkatanering"  # Vår felstavade sträng
a = list(a)  # Vi gör om den till en lista
a[6] = "e"  # Vi rättar
a = "".join(a)  # Sätter ihop till sträng igen
print(a)
```

```
Konkatenering
```

Kommandot `"".join` lägger ihop elementen i listan och placerar inget mellan elementen. Det som är mellan citattecknen placeras mellan elementen men vi har inget mellan dem. Testa gärna med att skriva in något annat mellan citattecknen tex. `"?".join(a)`

1.7 Skivor av listor och felmeddelande

Vi skivar upp listor på lite olika sätt för att repetera. Och fokuserar på hur felmeddelanden fungerar.

Om listan innehåller 5 element har vi som index

0	1	2	3	4
-5	-4	-3	-2	-1

Och om vi exempelvis väljer listan `L=[3,7,4,1,8]` så har vi

3	7	4	1	8
0	1	2	3	4
-5	-4	-3	-2	-1

[22]:
```
L = [3,7,4,1,8]
L[2:4]  # 2 inkl, till 4 exkl
```

[22]: [4, 1]

Vi får inget felmeddelande om index är utanför listan.

[23]:
```
L = [3,7,4,1,8]
L[2:7]  # 7 är utanför sista index som är 4
```

[23]: [4, 1, 8]

[24]:
```
L[-2]
```

[24]: 1

[25]:
```
L[:-2]  # Startar i 0 inkl går till -2 exkl
```

[25]: [3, 7, 4]

[26]:
```
L[-2:]  # Startar i -2 går till slutet
```

[26]: [1, 8]

Listkommandon ger inte alltid felmeddelande om du ger kommandon utanför indexets gränser. I vårt exempel har vi icke-negativa index från 0 tom. 4. Vad händer om man skriver

[27]:
```
L = [3,7,4,1,8]

L[5]
```

```
---------------------------------------------------------------------------
IndexError                                Traceback (most recent call last)
Cell In[27], line 3
      1 L = [3,7,4,1,8]
----> 3 L[5]

IndexError: list index out of range
```

Lägg märke till skillnaden. `L[2:7]` ger inte felmeddelande men `L[5]` ger felmeddelande!

Så om vi kanske tänkt helt fel och skriver

[ ]:
```
L = [3,7,4,1,8]
L[0:3000]
```

Så får vi inget felmeddelande.

Vad händer om båda gränserna anges fel (utanför listans index)?

[19]:
```
L = [3,7,4,1,8]
L[10:100]
```

[19]:
```
[]
```

Jo, vi får en tom lista. Var det det vi ville ha?

1.8 Vad finns i listan?

Man kan kontrollera om ett visst tal finns i en lista med hjälp av `in` och `not in`.

[20]:
```
L = [3,4,5]

3 in L
```

[20]:
```
True
```

[21]:
```
L = [3,4,5]

0 in L
```

[21]:
```
False
```

1.9 Slumplistor

Vi genererar listor med slumpmässigt valda naturliga tal (vi provade på detta när vi tittade på loopar). Det centrala kommandot är `random`. Vi använder *metoden* sample. Vi plockar värdena från listan M.

`random.sample(M,k)` betyder att vi tar k tal slumpmässigt från listan M.

[9]:
```
import random

M = [3,5,1,7]  # Tal tas slumpmässigt från denna lista
L=[]  # Här lägger vi de tal vi tagit

for i in range(1,11,1):  # Vi gör det 10 gånger
    L.append(random.sample(M,1))

print(L)
```

```
[[3], [5], [3], [7], [5], [1], [1], [7], [5], [3]]
```

Observera ovan att vi får listor i listor.

Se nedan. Vi får tal i intervallet 0 (inkl) till 10 (exkl). Vi får 8 tal. I detta kommando kan antalet genererade slumptal inte överstiga antalet tal i intervallet. Vi kan maximalt få 10 slumpade tal, alla olika. `range(0,10)` anger nu listan som vi får plocka tal ifrån. Slump utan återläggning.

[1]:
```
import random
L = random.sample(range(0,10),8)
print(L)
```

```
[7, 9, 3, 1, 8, 5, 2, 0]
```

Nedanstående fungerar inte eftersom vi ber om fler tal än vad som finns i mängden, det är utan återläggning.

[29]:
```
import random

L = random.sample(range(0,10),11)  # Fungerar ej
print(L)
```

```
---------------------------------------------------------------------------
ValueError                                Traceback (most recent call last)
Cell In[29], line 3
      1 import random
----> 3 L = random.sample(range(0,10),11)  # Fungerar ej
      4 print(L)

File ~/anaconda3/lib/python3.9/random.py:449, in Random.sample(self, population
 ↪k, counts)
    447 randbelow = self._randbelow
    448 if not 0 <= k <= n:
--> 449     raise ValueError("Sample larger than population or is negative")
    450 result = [None] * k
    451 setsize = 21        # size of a small set minus size of an empty list

ValueError: Sample larger than population or is negative
```

Slumpade tal med återläggning ges av `random.choices()`. Observera att det är plural choices, det finns ett annat kommando choice() som är singular.

[31]:
```
import random

L = random.choices(range(0,10),k=11)  # Fungerar med k=11 eftersom det är med
 ↪återläggning
print(L)
```

```
[8, 1, 2, 9, 4, 4, 0, 6, 4, 2, 9]
```

Observera att i choises måste man ange antalet som ett nyckelord, 'keyword', dvs du måste skriva k=11. I kommandot 'sample' ska du inte göra det, där är det *positionen* som används och du behöver bara skriva 11.

Slumptalen är naturligtvis inte alls slumptal utan helt deterministiska. De genereras med en algoritm som heter Mersenne-Twistor och ger en upprepning efter $2^{19937} - 1$ tal. Funktionerna kan således inte användas för tex. kryptering. För kryptering finns en modul som heter 'secrets'.

Vill man ha kontroll över sina slumptal, få samma slumptal varje gång man kör ett program så måste man ange ett slumpfrö. Anger man inte ett slumpfrö startar genereringen mha datorns systemtid. Vi anger nedan fröet 5 och kör choises() 2 gånger. Vi får exakt samma slumptal!

[33]:
```
import random

# Vi kör det en gång med fröet 5
random.seed(a=5)
L = random.choices(range(0,10),k=3)
print(L)

# Vi kör det en gång till med samma frö
random.seed(a=5)
L = random.choices(range(0,10),k=3)
print(L)
```

```
[6, 7, 7]
[6, 7, 7]
```

1.10 Listor i listor

Man kan ha listor som element i listor.

[11]:
```
a = [1,2,3]
b = ["a","b"]
a[1:1] = [b]  # observera skillnaden mot a[1:1]=b, se längre ner
# Vi har satt in listan b i listan a
print(a)

# Men vad är
a = [1,2,3]
b = ["a","b"]
a[1:1] = b  # observera skillnaden mot a[1:1]=[b], se ovan
# Vi har satt in element i listan b i listan a
print(a)
```

```
[1, ['a', 'b'], 2, 3]
[1, 'a', 'b', 2, 3]
```

Vi konstruerar direkt en lista med listor med listor.

[ ]:
```
L = [2,3, [4,5,['a','b']]]
```

Listan L ovan har en lista i sig och i den listan finns ytterligare en lista. Hur adresserar man ett element? Och hur många element har listan egentligen? Vi testar med kommandot len.

[12]:
```
L = [2,3, [4,5,['a','b']]]
len(L)
```

[12]: 3

Om vi vill ha tag i strängen `'a'` så anger vi först att det är elementet med index 2 i den yttre listan, däri är det element 2 i listan innanför och däri, innersta listan, element 0.

[ ]:
```
L = [2,3, [4,5,['a','b']]]
L[2][2][0]
```

För att få den första inre listan

[13]:
```
L[2]
```

[13]:
```
[4, 5, ['a', 'b']]
```

Och hela den inre listan i den

[14]:
```
L[2][2]
```

[14]:
```
['a', 'b']
```

Från elementet som är talet 5

[15]:
```
L[2][1:]
```

[15]:
```
[5, ['a', 'b']]
```

Ytterligare ett exempel

[16]:
```
K = [2,3, [4,5,['a','b', 3,4,5,7]]]
K[2][2][1:4]
```

[16]:
```
['b', 3, 4]
```

Och ett till

[1]:
```
a = [1,2,3]
b = [4,5,6]
c = [7,8,9]
L = [a,b,c]
print(L)
```

```
[[1, 2, 3], [4, 5, 6], [7, 8, 9]]
```

1.11 Kopiera listor (deep-copy)

Kopiering av listor är lite speciellt i förhållande till enkla variabler. Om vi skriver

```
x=1
y=x
print(y)
y=2
print(x)
```

så förväntar vi oss i den sista printsatsen att få svaret 1, inget annat; och så är det i detta fallet. Så fungerar det inte för listor. Det finns 2 sätt att kopiera, två sätt att tänka.

Vi betecknar listor med L respektive M. Om man skriver:

```
L=[1,2]  # original L
M=L  # kopia M
M.append(5) # vi lägger till 5 i kopian
print(L) # men vi har då lagt till 5 i originalet också!
```

så ger print-satsen: `[1,2,5]`. Som om vi hade ändrat originalet L när vi ändrade i kopian M. Satsen `M = L` innebär att M och L pekar på samma objekt, så ändrar du den ena ändrar du den andra. Man kan säga att L och M pekar på samma adress och där ligger en lista. `M = L` betyder peka på samma adress.

Naturligtvis kan man få en 'riktig' kopia, dvs en lista till som ser ut som L men som ligger någon annanstans i datorn och heter M. För att göra detta behöver man importera modulen copy och däri använda kommandot `copy.deepcopy(list)`.

[18]:
```
import copy

L = [1,2]  # Original
M = L  # Kopia
M.append(5)  # Ändrad kopia och ändrad original
print("original=", L, "ändrad kopia=", M)

# men

S = [1,2]  # Original
T = copy.deepcopy(S)  # Djup-kopia
T.append(5)  # Ändrad djup-kopia, ändrar inte original S
print("original=", S, "ändrad djup-kopia=", T)
```

```
original= [1, 2, 5] ändrad kopia= [1, 2, 5]
original= [1, 2] ändrad djup-kopia= [1, 2, 5]
```

1.12 Uppgifter

Uppgift 1

Använd `range()` för att skriva ut: a) De 100 första positiva heltalen. b) De 20 första positiva udda heltalen. c) De 25 första positiva jämna talen i fallande ordning. d) Samtliga talpar (R,B) som representerar de möjliga utfallen vid kast med två tärningar R och B, där B tärningen visar minst lika många prickar som R.

Lösningsförslag

Uppgift 2

Väv ihop 2 lika långa listor så att elementen tas omväxlande, varannan, från de 2 listorna. Konstruera listorna slumpmässigt med tal från 1 till 10 och 10 element i varje lista.

Lösningsförslag

Uppgift 3

Ändra ordningen på elementen i en lista. Säg att vi har en lista L, ändra om så att första elementet i listan hamnar sist i listan M, andra elementet i L hamnar näst sist i M osv. Det finns ett kommando

för detta `L.reverse()` men det ska du naturligtvis inte använda.

Lösningsförslag

Uppgift 4

Hur många tal ligger parvis i rätt ordning? Att tal ligger i rätt ordning parvis innebär att det med högre index är större än det med lägre index. I listan [7,11] ligger de i rätt ordning även om de inte direkt följer på varandra som 7 och 8 tex. Gå igenom en lista och testa talen parvis; först tal med index 0 och 1, sedan 1 och 2, sedan 2 och 3 osv. Anteckna antalet gånger talen ligger parvis rätt. Listan skapas slumpmässigt.

Lösningsförslag

Uppgift 5

Leta upp det största talet i en lista. I python finns ett kommando för just den uppgiften. Om L är en lista letar max(L) upp det största talet i listan L. Men nu ska du själv leta upp den genom att jämföra talen i listan med varandra. Slumpa listan och leta sedan upp det största talet i listan (utan att använda max). Listan ska innehålla 10 tal valda i intervallet 1-100.

Lösningsförslag

Uppgift 6

Generera långa listor och beräkna sannolikheten för att få tal omedelbart efter varandra som är lika. Listan L=[1,1,3,5,3,6,6,8,12,14,1,32,1,5,8,9,9,9] innehåller 4 händelser med 2 lika tal: (1,1), (6,6), (9,9) de 2 första och (9,9) de 2 sista.

Vi håller oss till tal i range(10). Svara med tabell. Slumptal skapas genom `s=random.randrange(0,10)` som ger ett tal from. 0 tom. 9. Man behöver inte göra listor men föreslår ändå att du börjar med att generera listor och använder dem (du ska öva på listor i detta avsnitt). Undersök listor med 100000 element och beräkna sannolikheten för längden 2 på sviter. Om du läst sannolikhetslära, beräkna det även teoretiskt.

Lösningsförslag

Uppgift 7

Skriv en lista med värden för sinus för heltal i grader från 0 till 90 i steg om 10 (0,10,20…90). Observera att Python är förinställt på radianer. Importera sin och pi från math.

Lösningsförslag

```
[ ]:
```

strings2

October 14, 2026

1 Mer om strängar

Strängar kan också adresseras som listor, utan att göra om dem till listor med hjälp av `list()`. Man kan dock inte ändra i dem men väl plocka ut en del av strängen. Variabler som inte kan ändras kallas för immutable. Strängar är variabler av typen immutable.

[1]:
```
S = "Ett äpple om dagen"
print(S[0])  # Vi kan addressera
```

```
E
```

[1]:
```
# Vi tar ut "Ett"

S = "Ett äpple om dagen"

print(S[0:3]) # Vi kan addressera
One = S[0:3]
print(One)
```

```
Ett
Ett
```

[2]:
```
S = "Ett äpple om dagen"

# Vi kan addressera och plocka ihop
Ny = S[0:4] + "päron " + S[10:13] + S[13:18]
print(Ny)
```

```
Ett päron om dagen
```

Man kan dock inte ändra i strängen på något enkelt sätt. Strängar är immutable.

[3]:
```
S = "Ett äpple om dagen"
S[0] = "A"  # Kommer inte ändra E till A
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Cell In[3], line 2
      1 S = "Ett äpple om dagen"
----> 2 S[0] = "A"
```

```
TypeError: 'str' object does not support item assignment
```

Observera skillnaden. Element i en sträng kan refereras till med hjälp av index men du kan inte ändra element genom att använda index.

Strängar kan användas utan att ges ett variabelnamn

[2]:
```
print("Solens gång"[1])
print("Solens gång"[3])
```

```
o
e
```

`.split()` delar upp en sträng. Den är förinställd på att dela vid mellanslag (inget argument behöver ges). Man kan också tex. dela vid kommatecken.

[5]:
```
a = "Solens gång"
x = a.split()
print(x)
```

```
['Solens', 'gång']
```

På liknande sätt kan strängar sättas ihop, tex. med mellanslag: `.join`

[5]:
```
x = ['Solens', 'gång']
y = " ".join(x)
print(type(y))
print(y)
```

```
<class 'str'>
Solens gång
```

1.1 Uppgifter

Uppgift 1

- Skapa en sträng: "God morgon alla människor"
- Dela upp strängen vid mellanslagen och skapa en lista med de 4 orden som sträng-element.
- Tag bort "alla" och "människor"
- Skriv ut listan.
- Lägg till "Johan" sist i listan.
- Skriv ut listan.
- Sätt ihop listans sträng-element till en sträng, skriv ut strängen.

Lösningsförslag

Uppgift 2

Låt användaren ange ett ord och kontrollera om det är en palindrom. Ej tillåtet använda kommandot .reverse() utan arbeta dig igenom listan själv.

Lösningsförslag

ProgramexN2

October 14, 2026

1 Exempel på program

Ger exempel på program utifrån det som tagits upp hitintills

1.1 Kvadratrotstabell

[1]:
```
import math as m

start = int(input("Var ska tabellen starta?"))
stop = int(input("Var ska tabellen sluta?"))
step = int(input("Hur stora steg ska tas?"))

print("x-värde", "roten ur x")

for i in range(start, stop+1, step):
    print("  ", i, "     ", m.sqrt(i))
```

```
Var ska tabellen starta? 1
Var ska tabellen sluta? 5
Hur stora steg ska tas? 1

x-värde roten ur x
   1       1.0
   2       1.4142135623730951
   3       1.7320508075688772
   4       2.0
   5       2.23606797749979
```

1.2 Sökning

Det finns flera olika sätt att sortera listor. Kan kanske tyckas vara en tråkig sysselsättning men mycket matematiskt arbete har lagts ner, och läggs ner, på att sortera så effektivt som möjligt. När du söker på internet är så att säga internet redan sorterat så det ska gå snabbt att söka. Ibland säger man att det inte är frågan om att använda sökmaskiner på internet utan sorteringsmaskiner. Google och Duck duck go osv. är egentligen sorteringsmaskiner.

I detta inledande fall ska vi ta en fix lista och söka upp det största talet, ett inledande steg i en sortering.

[ ]:
```
# Listan vi ska söka största talet i
L = [3, 7, 5]

# Vi gör en loop
large = L[0]  # Startläge, första talet är det största än så länge
for i in L:  # i antar värdena i listan, inget index behövs
    if i > large:  # Om vi hittar ett tal som är större så uppdatera variabeln␣
 ↪large
        large = i


print("Det största talet är", large)
```

Ändra i pythonprogrammet och testa lite olika tal (t.ex. alla tal lika, bara negativa tal).

Nu kan man lägga det största talet i en annan lista S och sudda det tal man fann och fortsätta tills listan L är tom, då har man en sorterad lista S!

1.3 Delbart med 3

Vi konstruerar ett program som undersöker om ett tal är delbart med 3.

[2]:
```
a = int(input("Ange ett heltal"))

if a//3 == a/3:
    print("Talet är delbart med 3")
else:
    print("Talet är inte delbart med 3")
```

```
Ange ett heltal 6

Talet är delbart med 3
```

Kontrollen sker genom att jämföra heltalsdivision `//` med vanlig division. Är de lika var talet delbart med 3, ingen rest förekom.

if-satsen ska ha något som är sant eller falskt. Jag har använt `==` som är ekvivalens. Om heltalsdivisionen är lika med divisionen så har uttrycket `a//3 == a/3` värdet `True` och satsen omedelbart efter utförs (och inget mer!), om det har värdet `False` görs det som står under `else`.

[ ]:
```

```

functions2

October 14, 2026

1 Funktioner

Funktioner i python kan ses som standardrutiner, ett eget litet program i programmet; en låda som gör en speciell sak och sedan skickar tillbaka resultatet till där man anropade funktionen. En funktion, en s.k. 'def', ses som en avgränsad låda som gör en specifik uppgift. Man skickar data till den och den skickar data tillbaka. Ibland kallas de också för subrutiner.

'Funktion' betyder i python inte bara funktion som du är van vid att möta den i matematiken utan är ett utvidgat funktionsbegrepp som mer liknar 'en sammanhållen grupp av beräkningar'. En python-funktion anges med det inledande ordet 'def'. 'def' är ett så kallat reserverat ord. Du bör inte använda det ordet till något annat i dina program.

1.1 Definition

Vi **definierar** funktionen $f(x) = 2x + 1$ i python genom att skriva just `def`. Funktionen börjar med `def` och slutar med att indraget upphör.

'def' är ett så kallat nyckelord (eng. keyword) det innebär att du inte bör använda den till något annat, tex. som en variabel.

[1]:
```
# Definiera en funktion
def f(x):
    y = 2*x + 1
    return(y)

print(f(3))  # Anropet till funktionen är f(3)
             # f(3) har erhållit värdet hos y med hjälp av return(y)
```

```
7
```

Funktionens definierande kommando är

```
def f(x):
    ett
    antal
    indragna
    satser
    return()
```

Funktionens namn är `f`. Parenteserna efter f omger *parametrar* som funktionen har. I detta fall har funktionen f parametern x.

Därefter ett kolon för att det kommer indragna satser, ett kod-block(svit).

`return()` för att skicka tillbaka något till anropet (ej nödvändigt). Det som skickas tillbaka är i exemplet värdet på `y` som i sin tur är resultatet av beräkningen `2*x + 1`.

Anropet till funktionen finns i `print(f(3))`. Anropet är `f(3)`. Värdet hos `f(3)` är värdet som beräknats och lagrats i `y` och skickas ut från def med hjälp av `return`; i just detta fall talet 7. Man kan säga att `f(3)` blir en variabel som tilldelas värdet av `y` genom `return(y)`.

Funktionen `f(x)` anropas med *argumentet* 3. Det är själva anropet som gör att funktionens kod körs; till skillnad från deklarationen. Att skriva själva def-en är en deklaration av den, men den körs inte då. Den körs då den anropas.

`x` i definitionen kallas för *parameter*.

Håll isär termerna argument och parameter annars orsakas missförstånd.

Python läser kod rad för rad. En def ska stå tidigt i kod, i regel direkt efter importerna.

Även om def-satser ska stå i början av koden så körs de **inte före** den egentliga koden; de deklareras först och körs sedan när någon sats **anropar** dem. Python bara läser def när den börjar läsa ditt program från rad 1, men def körs inte förrän den anropas.

Kör följande cell och den ger inget ut.

[3]:
```
# Definierar en funktion; den körs inte när du klickar på symbolen för 'run',
# ger inget ut även om det står en print-sats.
def f(x):
    print("Hej")
    return(2*x + 1)
```

I och med att def är en deklaration så är det inte effektivt att ha den t.ex. i en loop; python behöver bara ha funktionen deklarerad 1 gång, då i början av programmet. I en loop deklareras den flera gånger, vilket är resurskrävande.

Antalet argument måste stämma med antalet parametrar annars ges felmeddelande (vi ska se strax hur man kommer förbi detta).

Vi börjar med för många argument.

[6]:
```
def f(x):
    y = 2*x + 1
    return(y)

print(f(3,2)) # Vi skickar 2 argument men det finns bara en parameter
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Cell In[6], line 5
      2     y = 2*x + 1
      3     return(y)
----> 5 print(f(3,2))
```

```
TypeError: f() takes 1 positional argument but 2 were given
```

Vi skickar 1 argument men det finns 2 parametrar.

[7]:
```
def f(x,y):
    z = 2*x + y
    return(z)

print(f(3)) # Vi skickar 1 argument men det finns 2 parametrar
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Cell In[7], line 5
      2     z = 2*x + y
      3     return(z)
----> 5 print(f(3))

TypeError: f() missing 1 required positional argument: 'y'
```

1.2 Exempel

1.2.1 Kuben av ett tal

Vi definierar en funktion som beräknar kuben av ett tal och använder den i en loop för att beräkna summan av kuber. (Inte mycket lönt, men av didaktiska orsaker)

[9]:
```
# Definierar en funktion för kuben av ett tal; inget utförs
def kub(n):
  return(n**3)

summakub = 0  # Initiering av variabel
print("i   summa")  # Rubrik för tabell

for i in range(1,21,1):  # Från 1 till 20
  summakub = kub(i) + summakub  # Anrop av funktionen kub(); nu utförs den
  print(i, "   ",summakub)
```

```
i   summa
1      1
2      9
3      36
4      100
5      225
6      441
7      784
```

```
8      1296
9      2025
10      3025
11      4356
12      6084
13      8281
14      11025
15      14400
16      18496
17      23409
18      29241
19      36100
20      44100
```

1.2.2 Produkten av två tal

En funktion som beräknar produkten av två tal som matas in i programmet. Skickar nu 2 tal till en funktion, inte bara 1 som tidigare.

[10]:
```
print("Beräknar produkten av två tal")

def prod(x,y):  # Funktionen definieras, den ska ha 2 tal
  return(x*y)



a = float(input("Ange det första talet "))
b = float(input("Ange det andra talet "))
print("Produkten är ", prod(a,b))  # Anrop
```

```
Beräknar produkten av två tal

Ange det första talet  4
Ange det andra talet  6

Produkten är  24.0
```

1.2.3 Räta linjen

Programmet skickar 2 tal tillbaka, inte bara 1. När 2 tal, eller fler, skickas tillbaka med return så kan värdena tas fram enskilt genom att som vid listor ange index.

Vi använder $y = kx + m$. Och med $y_1 = kx_1 + m$ och $y_2 = kx_2 + m$ för två punkter på linjen. Vi erhåller för riktningskoefficienten

$$k = \frac{y_1 - y_2}{x_1 - x_2}$$

och interceptet

$$m = y_2 - k \cdot x_2$$

[4]:
```
"""
Programmet bestämmer riktningskoefficient och
intercept för en rät linje genom två angivna punkter.
(på detta sätt kan man skriva övergripande kommentarer)
"""

# Första punkten
L1 = [1.0,3.0]  # (x1,y1)
# Andra punkten
L2 = [3,11.]  # (x2,y2)
# Kan ändras till input() senare när programmet fungerar


def km(l1,l2):
  k = (l1[1]-l2[1])/(l1[0]-l2[0])  # k = (y1-y2)/(x1-x2)
  m = l2[1]-k*l2[0]  # y2 = kx2 + m -> m = y2 - kx2

  return(k,m)  # Resultat, 2 tal, blir automatiskt en tiple; fungerar ungefär␣
 ↪som en lista


values = km(L1,L2)  # Anrop

# De enskilda värdena i values kan nås med index: values[0] är första elementet
print("k =", values[0], "m =", values[1])
```

```
k = 4.0 m = -1.0
```

Vi gör en annan variant på föregående program om riktningskoefficient. Vi lagrar de 2 värdena i en lista och skickar listan som 1 element. Jämför.

[4]:
```
#Första punkten
L1 = [1.0,3.0]  # (x1,y1)
#Andra punkten
L2 = [3,11.]  # (x2,y2)
# Kan ändras till input() senare


def km(l1,l2):
  k=(l1[1]-l2[1])/(l1[0]-l2[0])  # k = (y1-y2)/(x1-x2)
  m=l2[1]-k*l2[0]  # y2 = kx2 + m -> m = y2 - kx2
  M=[k,m]  # NY Vi lägger nu de två talen i en lista
  return(M)  # NY Vi skickar nu ut listan


values = km(L1,L2)  # Anrop. values tilldelas M
# values är nu verkligen en lista
print("k =", values[0], "m =", values[1])
```

```
k = 4.0 m = -1.0
```

Precis som i matematiken kan man arbeta med sammansatta funktioner. Skrivsättet liknar det vi är vana vid. Definiera två funktioner.

[7]:
```
def f(x):
    y = 2*x + 1
    return(y)

def g(x):
    y = x**3
    return(y)

fg = f(g(2))
gf = g(f(2))

print("f(g(2))=", fg)

print("g(f(2))=", gf)
```

```
f(g(2))= 17
g(f(2))= 125
```

Definitionen av funktionen kan, som nämnts, inte ligga efter anropet. Det genererar felet NameError.

[5]:
```
print(ff(5))  # Kan inte ligga före definitionen/deklarationen

def ff(x):
    y = 2*x + 1
    return(y)
```

```
---------------------------------------------------------------------------
NameError                                 Traceback (most recent call last)
Cell In[5], line 1
----> 1 print(ff(5))  # Kan inte ligga före definitionen/deklarationen
      3 def ff(x):
      4     y = 2*x + 1

NameError: name 'ff' is not defined
```

1.2.4 Rekursiv funktion

Att en funktion är rekursiv innebär att den anropar sig själv. Vi studerar den rekursiva funktionen $a_n = 3a_{n-1}$ med $a_0 = 5$. Denna kan också skrivas som $f(n) = 3f(n-1)$.

[7]:
```
"""
Löser en rekursiv ekvation
```

```
a_(n)-3a_(n-1)=0
a_(n)=3a_(n-1)
"""


def rekur(n):
  if n == 0:
    return 5  # a0 = 5
  else:
    print(n) # för felsökning
    return 3*rekur(n-1)


# Beräknar termen med index n
# n=3 ger 5*3*3*3=135 dvs 5*3**3
# n=12 ger 2657205 dvs 5*3**12

print('vilken term ska beräknas?\n')
a = int(input('ange ordningsindex '))
term = rekur(a)
print('termens storlek är', term)
```

```
vilken term ska beräknas?


ange ordningsindex  1
```

```
1
termens storlek är 15
```

Vi löser differensekvationen:

$$a_{n+2} - 2a_{n+1} + 2a_n = 0$$

Vi löser ut a_n

$$a_{n+2} = 2a_{n+1} - 2a_n$$

Vi skriver om den med $m = n + 2$.

$$a_m = 2a_{m-1} - 2a_{m-2}$$

Vi måste också ha 2 startvärden: $a_{-1} = 4$, $a_{-2} = 5$, då vi börjar på $m = 0$.

[8]:
```
def rekur(m):
  if m == -1:
    return 4  # a(-1) = 4
  elif m == -2:
    return 5  # a(-2) = 5
  else:
```

```
    am = 2*rekur(m-1)-2*rekur(m-2)
    return am


print('vilken term ska beräknas?\n')
a = int(input('ange ordningsindex '))
term = rekur(a)
print(a,'termens storlek är', term)
```

```
vilken term ska beräknas?

3 termens storlek är -16
```

1.3 Variabelt antal argument

I exemplet Räta linjen skickas listor till funktionen km(). Funktionen km() är också beredd på att ta emot listor. När funktionen är färdig så skickas 2 tal, k och m, som en tipel (eller lista) och kan nås via index.

Men det finns varianter på detta.

Låt oss först definiera *positionsbestämda argument* som att det är argument som tilldelas värden utifrån sina positioner. Första parametern får värdet som anges som första argument, andra parametern får värdet som anges av andra argumentet osv.

[4]:
```
def f(a, b):
    adivb = a/b
    return (adivb)


print(f(5, 10))  # a tilldelas 5 och b tilldelas 10

print(f(10, 5))  # a tilldelas 10 och b tilldelas 5
```

```
0.5
2.0
```

Packa upp: Att ha data i en lista och skicka till en funktion som har flera parametrar. Lite slarvigt innebär det att elementen i listan som skickas blir argument till parametrarna. För att ange för python att den ska packa upp listan så skriver man * framför argumentet; i nedanstående fall `*vikter`.

[1]:
```
# Tre tal adderas med olika vikter

def viktsumma(a,b,c):
  s = 1*a + 5*b + 10*c
  return s


vikter=[0.1,0.3,0.6]
```

```
print(viktsumma(*vikter))  # Packa upp listan vikter o dela ut till parametrarna
```

```
7.6
```

När `viktsumma()` anropas i print-satsen har vi listan `vikter` som argument. Men funktionen är inte inställd på att ta emot en lista (vilket bara är 1 argument) utan är inställd på att ta 3 argument eftersom den har 3 parametrar. Men om python packar upp listan och tilldelar parametern a värdet i vikter[0], tilldelar parametern b värdet i vikter[1] osv, så blir det som jag vill. (naturligtvis kan man packa upp listan själv)

Packa ihop: Den andra möjligheten som finns är att de argument som ges tas emot av funktionen och där packas ihop till en lista som sedan funktionen använder. Antalet argument är inte bestämt. I regel är det dock 1 parameter, i alla fall för varje lista som ska skapas. Det behöver således inte vara bestämt hur många argument som ska skickas!

Egentligen är det inte en lista utan en tipel. Men för vår användning fungerar den på samma sätt. Du kan använda index för att få tag i de olika elementen i tipeln.

[16]:
```
"""
Tecknet * vid parametern gör om de 5 argumenten till en lista som
sedan används för en loop
"""

def func(*args):  # De 5 argumenten packas ihop till en lista som heter args
    print("Nu har vi gjort en tipel ", args) # Vi skriver tipeln.
    print("Vi skriver ut element 3: ", args[2])

    s = 0
    for a in args: # Vi loopar igenom tipeln
        print(a)
        s = s + a
    return(s)

dd = func(2,3,1,5,3)  # Skickar 5 argument, inte en lista eller tipel
print("summan av tipelns element är", dd)
```

```
Nu har vi gjort en tipel  (2, 3, 1, 5, 3)
Vi skriver ut element 3:  1
2
3
1
5
3
summan av tipelns element är 14
```

Vill bara påminna om att en tipel är som en lista men du kan inte ändra i den och den skrivs med () och inte [].

Men jag kan själv packa in dem i en lista.

[12]:
```
def func(args):  # Tar nu 1 argument
    s = 0
    for a in args:
        s = s + a
    return(s)

dd = func([2,3,1,5,3])  # Skickar inte 5 argument, utan en lista, vilket är 1␣
 ↪argument.
print("summan av listans element är", dd)
```

```
summan av listans element är 14
```

Kom ihåg:

Stjärna * vid anrop (argument) packar upp: anrop(*upp)

Stjärna * vid def (parameter) packar ihop: def(*ihop)

1.4 Förinställd parameter

I många sammanhang kan man vilja arbeta med en funktion på olika sätt. Man kan då använda *förinställda* parametrar (default). Jag vill att ett program ska producera enbart slutresultat om den anropas på vanligt sätt. Om jag däremot anger ett extra argument så vill jag tex. att den skriver ut mer information.

Man kan ge en parameter ett visst värde i själva deklarationen av funktionen. Om man då anropar funktionen med så att säga för lite argument så kommer den saknade parametern att få värdet som anges i deklarationen.

[8]:
```
# s har värdet True förinställt, så om inget argument anges så är s=True

def summa(a, b, c, s=True):
    if s != True:
        print("speciellt anrop\n", "a =", a, "b =", b, "c =", c)

    return (a+b+c)


slutsats = summa(11, 12, 13)
print("vanligt anrop ", slutsats)

slutsats = summa(11, 12, 13, False)
print(slutsats)
```

```
vanligt anrop  36
speciellt anrop
 a = 11 b = 12 c = 13
36
```

Förinställt värde är mycket vanligt. Om vi ser tillbaka på kommandot `range(start,stopp,steg)` så kan vi vid ett anrop av denna funktion skriva `range(8)` som anger att stopp är 8. start och steg

antar då de *förinställda* värdena 0 respektive 1. Det innebär att i deklarationen av range så står det (i princip)

```
def range(start=0, steg=1, stopp)
```

1.5 Nyckelord

Om man inte vill ha positionsbestämda variabler så kan man använda nyckelord. Nyckelord är parametrarna. Man anger helt enkelt i anropet vad parametern heter och vilket värde den ska ha; ordningen spelar då ingen roll. Nackdelen är att man måste hålla reda på nyckelorden. Ibland är dock nyckelorden enkla att minnas som tex. color eller size. Detta kallas på engelska för "keyword arguments" och förkortas kwargs i pythons dokumentation.

[17]:
```
def f(a,b):  # Här i deklarationen är det ingen skillnad.
    adivb = a/b
    return(adivb)

print(f(a=5,b=10))  # a tilldelas 5 och b tilldelas 10

print(f(b=10,a=5))  # a tilldelas 5 och b tilldelas 10. Att vi skrivit dem i␣
 ↪annan ordning
                    # spelar ingen roll.
```

```
0.5
0.5
```

Ytterligare ett exempel på användningen av nyckelord.

[2]:
```
def kostnad(vikt, kilopris, rabatt):
    s = vikt*kilopris*(1-rabatt/100)
    return(s)

print(kostnad(rabatt=20, kilopris=35, vikt=2.3))
# Argumenten inte i ordning, men de har namn som är lätta att komma ihåg
```

```
64.4
```

Det är tillåtet att använda både positionsbestämda parametrar tillsammans med nyckelord men positionsparametrarna måste stå först, därefter nyckelord.

Om position används efter nyckelord erhålls felmeddelande: "positional argument follows keyword argument"

[5]:
```
def f(a,b,c,d,e):
    print(a,b,c,d,e)
    adivb = a/b
    return(adivb)

print(f(5,10, d=1, c=2, e=3))  # Position för 5 och 10 resten nyckelord, OK.
```

```
#print(f(5,10, d=1, c=2, 3))  # Försöker med position på sista variabeln,␣
 ↪fungerar ej
```

```
5 10 2 1 3
0.5
```

Position och packa ihop kan kombineras. Här låter vi de 2 första argumenten få var sin parameter. Övriga argument, talen 1 och 2, packas ihop till en lista som då får de kvarvarande 2 elementen. Vi adresserar genom index.

[2]:
```
def f(a,b,*c):
    print(a,b,c[0],c[1])
    adivb = a/b
    return(adivb)

print(f(5,10,1,2))  # Position för 5 och 10 resten packas ihop vid def.
```

```
5 10 1 2
0.5
```

Avslutar med 2 viktiga påpekanden:

1. Placera inte import av moduler inuti loopar. Moduler ska bara importeras en gång i ett program.
2. Placera inte def inuti loopar. Funktioner ska bara definieras en gång i ett program.

1.6 Uppgifter

Uppgift 1

Det aritmetiska medelvärdet är alltid större än det geometriska. För två tal x och y gäller

$$\frac{x+y}{2} \geq \sqrt{xy}.$$

Skriv en funktion(def) som har x och y som parametrar och som skickar tillbaka det aritmetiska och geometriska medelvärdet. Programmet skriver sedan ut båda medelvärdena så de kan jämföras. Användaren ska ange x och y.

Lösningsförslag

Uppgift 2

Skriv 2 funktioner som beräknar area respektive omkrets av en rektangel. Funktionernas parametrar ska vara rektangelns bredd och höjd och utdata ska vara arean respektive omkretsen. Användaren anger bredd och höjd.

Lösningsförslag

Uppgift 3

Skapa en lista av tal jämnt fördelade mellan 0 och 2π. Skapa en funktion(def) som beräknar det

kvadratiska trigonometriska medelvärdet av talen i listan:

$$\frac{1}{N}\sum_{i=1}^{N}\sin^2(x_i)$$

Vad blir det för värde då N är stort?

Lösningsförslag

Uppgift 4

Skriv en rekursiv funktion som beräknar det n:te talet i talföljden som ges av $a_{n+1} = 4a_n - 1$ med begynnelsevillkoret $a_0 = 1$. Funktionens parametrar ska vara talets plats i talföljden och begynnelsevillkoret. Utdata ska vara själva talet. Funktionen ska vara rekursiv, dvs. anropa sig själv inne i funktionen.

Lösningsförslag

Uppgift 5

Skapa en rekursiv funktion som beräknar fakulteter. Tex. är $7! = 7 \cdot 6 \cdot 5 \cdot 4 \cdot 3 \cdot 2 \cdot 1$. Rekursiviteten kan beskrivas genom funktionen $(7! =) f(7) = 7 \cdot f(6)$. Och $f(1) = 1$. Vi har rent allmänt att $f(n) = n \cdot f(n-1)$.

Lösningsförslag

[ ]:

plot2

October 14, 2026

1 Att rita grafer (plot)

De förslag på hur man ritar grafer som framförs här grundar sig på matplotlib.pyplot. Det kallas att plotta med Pyplot. Det ger snabbt resultat. Om man vill kontrollera ritandet mer i detalj så rekommenderar python.org att man använder det objekt orienterade skrivsättet i stället. Vi behandlar inte det här utan det kommer att finnas på en annan plats.

Här på jupyter ritas graferna på ett visst sätt med de inställningar som finns. Ritar du dem på replit.com eller på din egen dator kommer det att se annorlunda ut.

1.1 Mallen

Vi ska försöka rita $y = sin(x)$. För att kunna rita grafer måste vi importera moduler. Vi har konstaterat att vi i grundutförandet inte kan göra `sin(1.2)`. Men om vi importerade math-modulen så gick det bra. Men för att kunna rita vår funktion så önskar vi oss mer; precis vad ska vi strax se.

För att kunna rita för hand behöver vi en värdetabell och det behöver python också. Värdetabell:

x	sin(x)
0.1	0.0998
0.2	0.1987
0.3	0.2955
0.4	0.3894

Så vi måste ha en lista med x-värden tex. `Lx=[0.1,0.2,0.3,0.4]`. Detta kan `range()` hjälpa oss med.

Men sedan vill vi beräkna sinusvärden för alla talen i listan. Då kan man göra en loop med `sin(Lx[i])`. Men man har fixat detta problem och lagt det i en modul så man kan direkt skriva sinus av en lista och få sinus för alla elementen i listan. Kommandot `sin(Lx)` är tillåtet och python gör en ny lista med alla sinusvärdena. Vi behöver en modul som heter numpy (numerical python). Vi testar detta kommando först innan vi behandlar hur värdetabellen ska ritas.

Man skulle kunna säga att det fungerar så här:

$\sin(Lx) = \sin([0.1, 0.2, 0.3, 0.4]) = [\sin(0.1), \sin(0.2), \sin(0.3), \sin(0.4)] =$
$[0.0998, 0.1987, 0.2955, 0.3894] = Ly$

[2]:
```
import numpy as np

Lx = [0.1,0.2,0.3,0.4]
Ly = np.sin(Lx)  # Beräknar alla y-värden på en gång
print(Lx,Ly)
print(type(Lx))  # Lx är en lista
print(type(Ly))  # Ly är en ny datatyp ndarray
```

```
[0.1, 0.2, 0.3, 0.4] [0.09983342 0.19866933 0.29552021 0.38941834]
<class 'list'>
<class 'numpy.ndarray'>
```

Man har gjort det ännu smidigare genom ett kommando man infört. Det kommandot heter `linspace()`. linspace skapar en datatyp som heter ndarray eller bara array (används för vektorer). array har inte samma egenskaper som en lista. Skillnaden syns om man beräknar tex. `2*L` där L är en lista respektive en array. Om L är en lista gäller konkatenering. Om L är en array gäller att varje element multipliceras med 2. Precis så fungerar vektorer: $2(3,4) = (6,8)$

Skilj på linspace och lista.

[1]:
```
import numpy as np

L1 = np.linspace(0,4,5)  # array, (start, stopp, antal)
L2 = [0,1,2,3,4]  # Lista
L3 = np.array(L2)  # Omvandlar listan till array

print(L1,L2)
print("\n")
print(2*L1, type(L1))  # array; varje element multipliceras med 2
print(2*L2, type(L2))  # lista; skapar 2 listor som konkateneras
print(2*L3, type(L3))  # array
```

```
[0. 1. 2. 3. 4.] [0, 1, 2, 3, 4]


[0. 2. 4. 6. 8.] <class 'numpy.ndarray'>
[0, 1, 2, 3, 4, 0, 1, 2, 3, 4] <class 'list'>
[0 2 4 6 8] <class 'numpy.ndarray'>
```

ndarray står för n-dimensional array, en datatyp. Observera att array inte har kommatecken som avskiljare av elementen utan endast mellanslag. Det är dock hakparentser i båda fallen.

Med ord kan det sammanfattas som: * 2*lista: ger konkatenering* 2*array: ger elementvis multiplikation* linspace ger en array

Skrivsättet blir väldigt suggestivt. Man kan göra samma sak med flera olika tal på samma gång. Vi tittar närmare på linspace.

[2]:
```
import numpy as np

x=np.linspace(-2*np.pi,2*np.pi,10)
```

```
print(x)
```

```
[-6.28318531 -4.88692191 -3.4906585  -2.0943951  -0.6981317   0.6981317
  2.0943951   3.4906585   4.88692191  6.28318531]
```

Kommandot `linspace()` ska ha start och stopp och antal värden `linspace(start, stopp, num)`. I detta fall har jag använt intervallet från -2π till 2π. Allra sist står antalet (`num`) tal som jag vill ha. I detta fall 10 stycken.

Observera att stopp ingår, se nedan.

[2]:
```
import numpy as np

x=np.linspace(-2,2,10)
print(x)
```

```
[-2.         -1.55555556 -1.11111111 -0.66666667 -0.22222222  0.22222222
  0.66666667  1.11111111  1.55555556  2.        ]
```

Vi gör en värdetabell, både x och y

[4]:
```
import numpy as np

x = np.linspace(-2*np.pi,2*np.pi,10)  # x är en array
print("x-värden",x)
y = np.sin(x)  # nästan samma skrivsätt som i matematiken
print("y-värden",y)
```

```
x-värden [-6.28318531 -4.88692191 -3.4906585  -2.0943951  -0.6981317   0.6981317
  2.0943951   3.4906585   4.88692191  6.28318531]
y-värden [ 2.44929360e-16  9.84807753e-01  3.42020143e-01 -8.66025404e-01
 -6.42787610e-01  6.42787610e-01  8.66025404e-01 -3.42020143e-01
 -9.84807753e-01 -2.44929360e-16]
```

Nu har vi vår värdetabell klar. Nu behöver vi ladda en modul som sköter om själva plottandet.

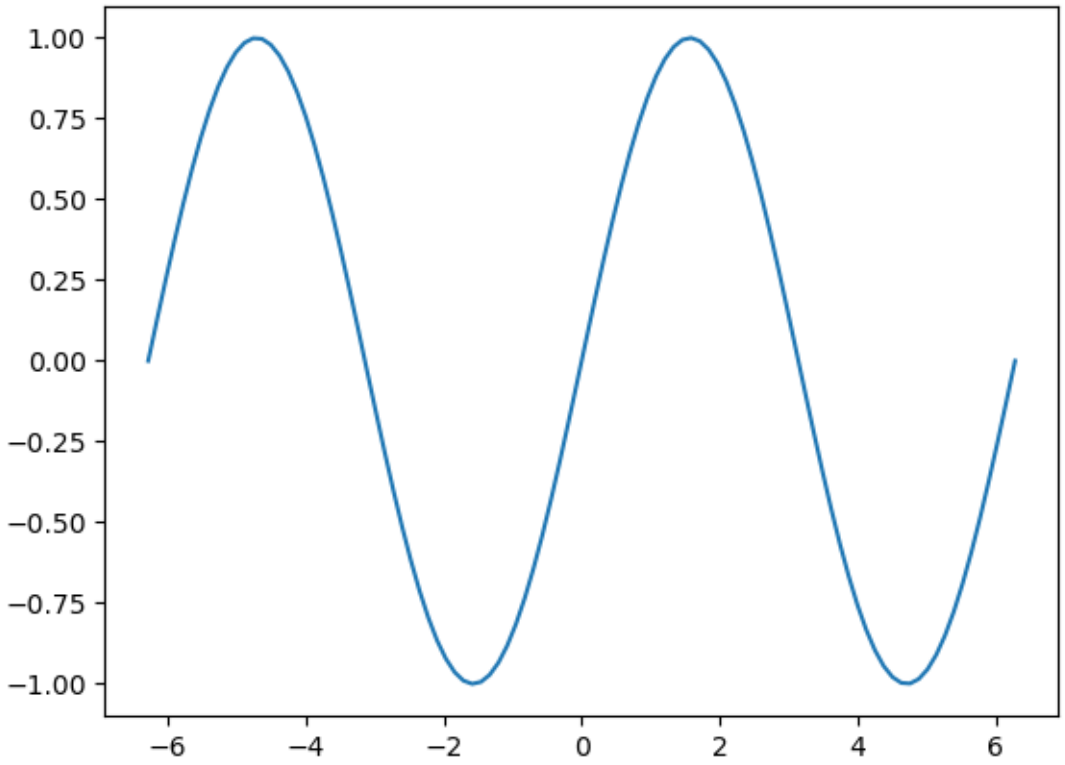
Modulen som behöver importeras, heter `matplotlib.pyplot`. De nya kommandona framgår av koden.

[4]:
```
# Mall för enkel plottning

import matplotlib.pyplot as plt
import numpy as np

# Värdetabell enligt tidigare
x = np.linspace(-2*np.pi, 2*np.pi, 100)
y = np.sin(x)

# Nu ritar vi!
plt.plot(x,y)
plt.show()
```



Python justerar själv koordinatsystemet om man inte säger något, dock finns det kommando för det också.

Om du skrivit raderna i en textfil, tex. på replit.com måste du även ange kommandot `plt.show()` annars får du ingen graf att själv titta på. (jupyter klarar sig utan den, men lägger till en rad i utskriften om den inte är med)

Följden av kommandon (import, x=, y=, plot…) används ofta som en mall, sedan är det bara att fylla i och ändra efter behov.

Vid plottning är det vanligast med array, men även listor förekommer. array är bekvämast då man ska göra samma beräkning för alla x-värden för att få y-värdena.

1.2 Utsmyckning

Naturligtvis kan man själv ändra koordinatsystem, rita flera kurvor i samma diagram, ändra färger på kurvorna, lägga in förklarande text osv.

Exempel följer. Observera att i anropet på funktionen plt.plot() används nyckelord, vilket vi behandlat i 2.4 om funktioner.

[1]:
```
import matplotlib.pyplot as plt
import numpy as np

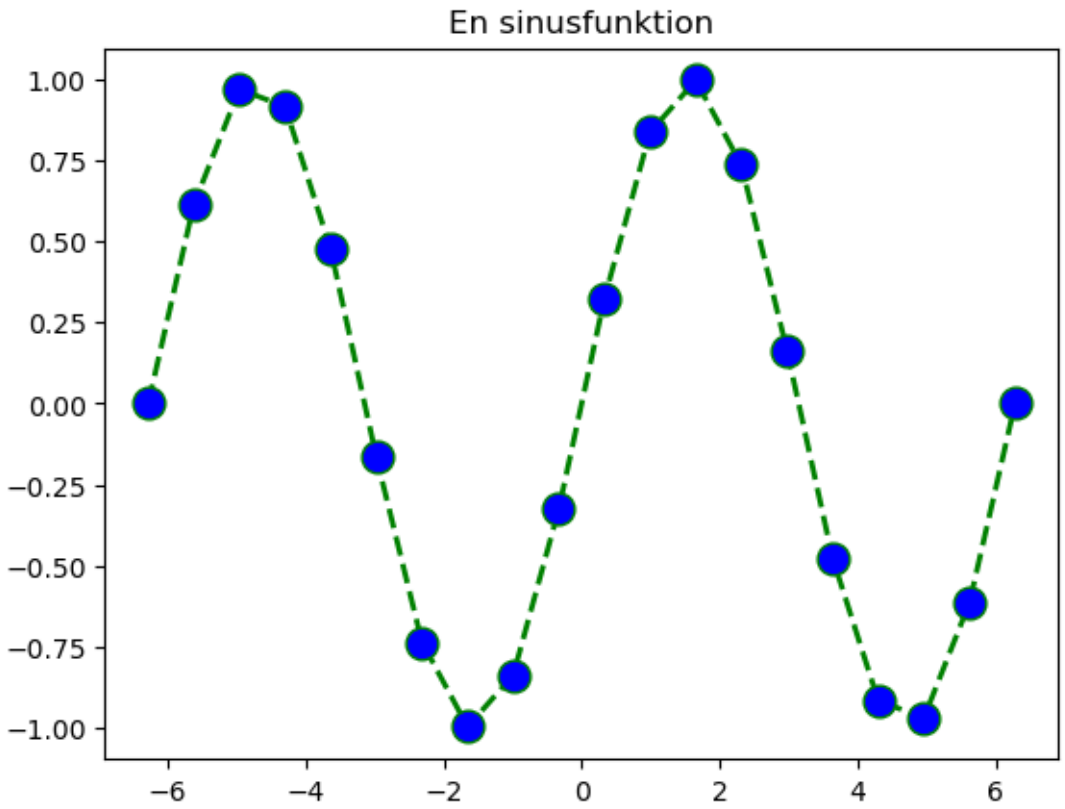
# Värdetabell
```

```
x = np.linspace(-2*np.pi, 2*np.pi, 20)  # array
y = np.sin(x)  # array

# Rita med utsmyckning
plt.plot(x,y, color="green",
         linestyle='dashed',
         marker='o',
         markerfacecolor='blue',
         markersize=12,
         linewidth=2)

plt.title('En sinusfunktion')

plt.show()
```



Observera hur hela kommandot plot() är uppdelat på flera olika rader, det ökar läsbarheten. Det är tillåtet att göra så och python väntar på den avslutande högerparentesen.

- `color="green"` betyder att kurvan ritas med grön färg
- `linestyle="dashed"` betyder att kurvan är streckad
- `marker="o"` innebär att datapunkterna markeras med en cirkelskiva
- `markerfacecolor="blue"` gör att cirklarna är fyllda med blått (inte gröna som kurvan)

- `markersize=12` anger storleken på markören
- `linewidth=6` anger bredden på kurvan

Som du kanske inser från tidigare avsnitt så har flera av parametrarna (color, linestyle osv) egna default (förinställda) värden som används om du inte anger något. Vi har tex. angett att color ska ha värdet "green", men om vi inte skickar argumentet "green" så finns det ett förinställt värde som python använder.

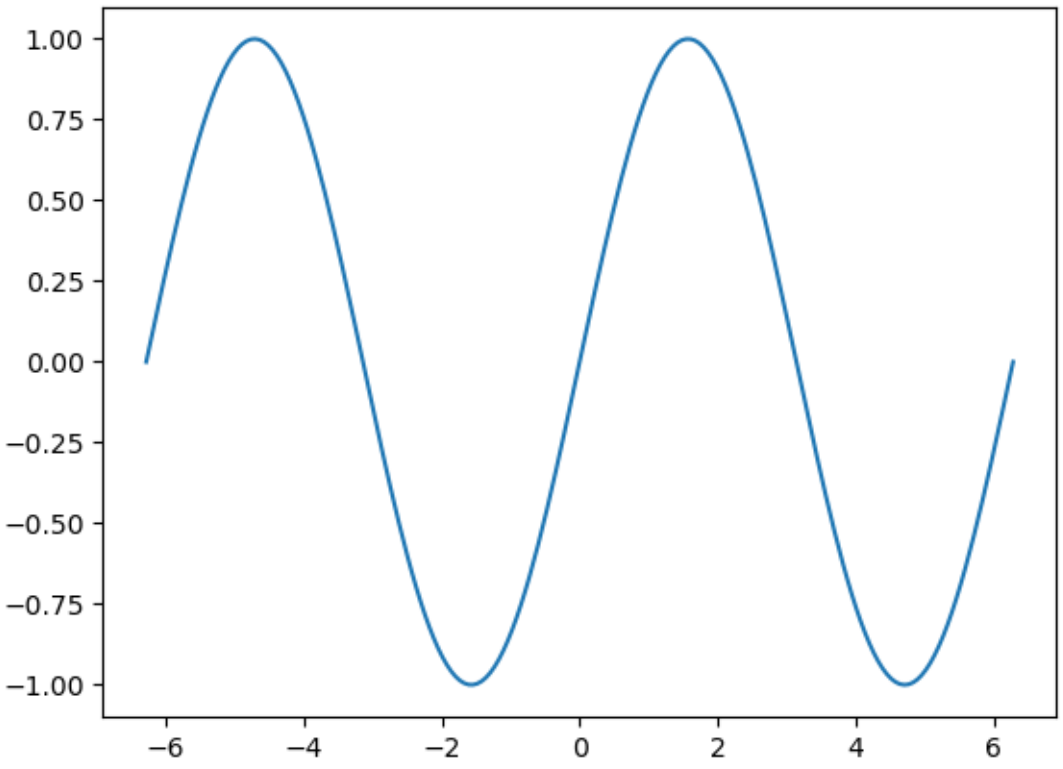
Experimentera. Observera stavningen av färg, det är color inte colour. Observera också att det är strängar som ska anges så citattecken måste vara med.

På replit.com finns det möjlighet att spara figuren genom att klicka på en ikon i figuren. Men man kan också skriva till ett kommando för det `plt.savefig()`.

[2]:
```
import matplotlib.pyplot as plt
import numpy as np


x=np.linspace(-2*np.pi, 2*np.pi, 200)
y=np.sin(x)
plt.plot(x,y)

plt.savefig('plot.png')  # Fler format finns
plt.show()
```



På replit.com syns grafen till höger (i standardinställningen) och en fil med namnet plot.png skapas och läggs bland filerna till vänster där main.py ligger. Denna kan du sedan ladda ner. Flera olika bildformat fungerar.

1.3 Två kurvor i samma graf

Vi har ritat endast en kurva i grafen, nu ritar vi 2.

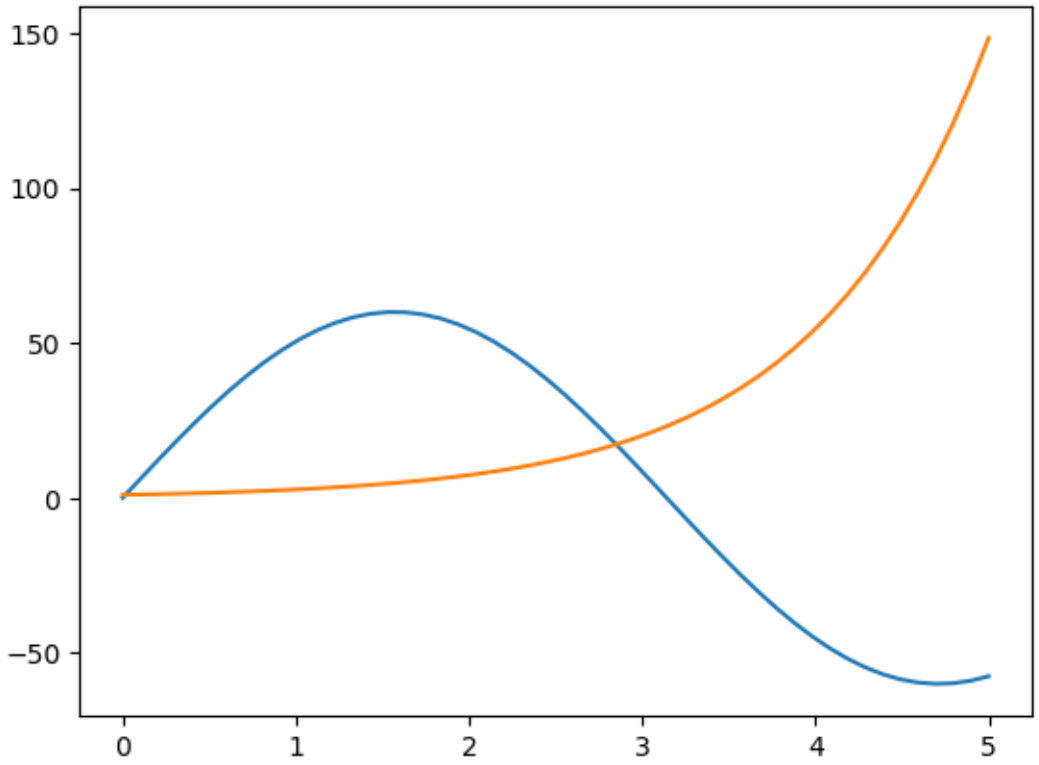
[11]:
```
import matplotlib.pyplot as plt
import numpy as np

# Kurva 1
x = np.linspace(0, 5, 50)
y = 60*np.sin(x)
plt.plot(x,y)

# Kurva 2
x = np.linspace(0, 5, 50)
y = np.exp(x)
plt.plot(x,y)

#Ritas i samma graf!

plt.show()
```



Vad är det som gör att det blir två kurvor i en graf och inte två grafer med en kurva i varje? Det som är signifikant är att kommandot plt.show() kommer efter att vi gett två kommandon plt.plot(). Om vi ger kommandot plt.show() omedelbart efter ett plt.plot() så får vi två grafer.

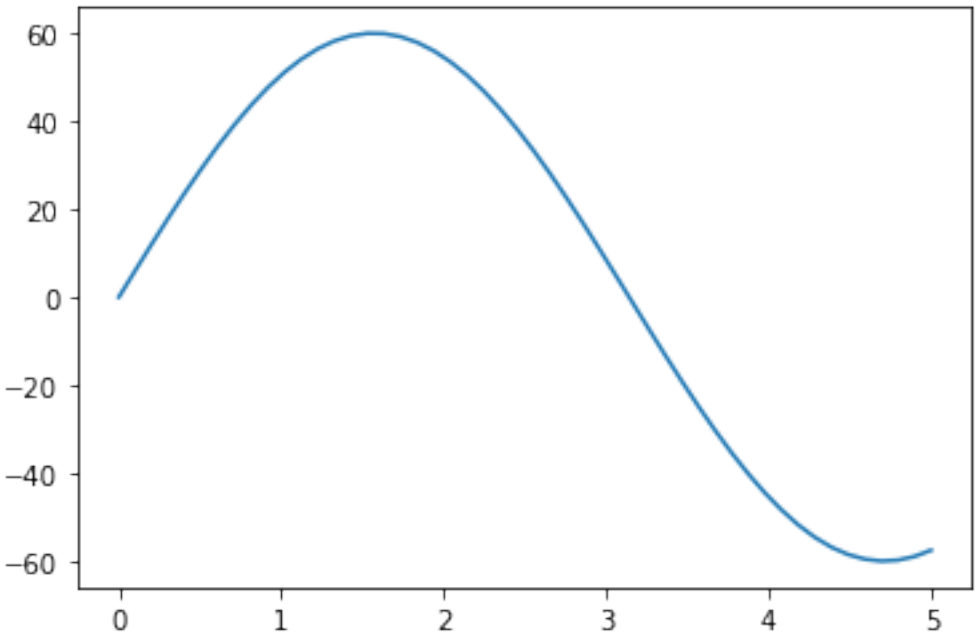
Se nedan för 2 åtskilda plottar. På jupyter visas båda plottarna direkt. På replit.com måste du stänga den första för att se den andra. Om du sitter vid en vanlig terminal på en dator måste du också stänga den första grafen för att få den andra. Beroende på val av IDE kan det kanske variera.

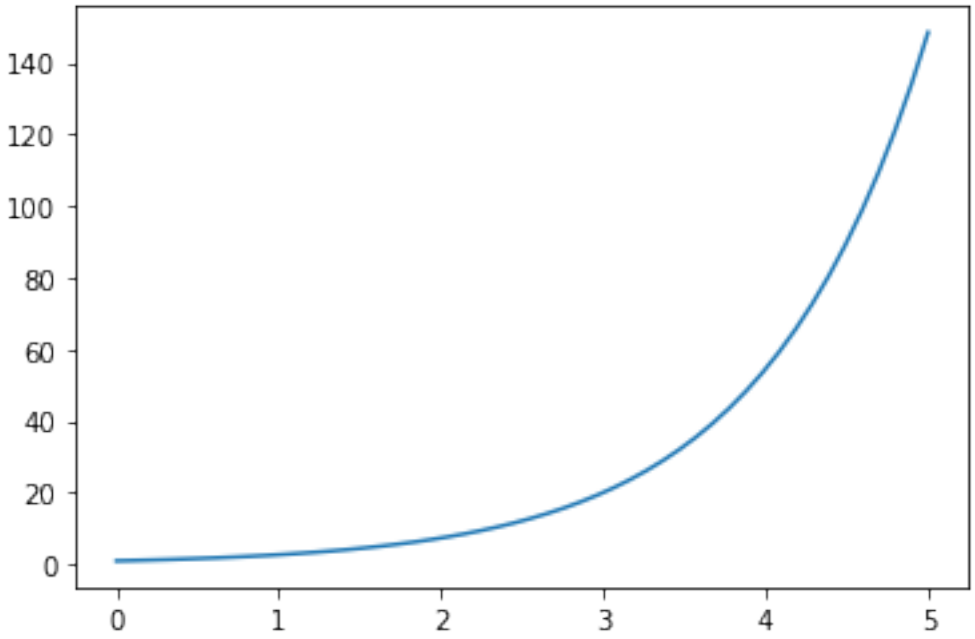
I vilket fall så kan man alltid spara dem (plt.savefig("namn"); innan plt.show()) en och en.

[5]:
```
import matplotlib.pyplot as plt
import numpy as np

# Kurva 1
x = np.linspace(0, 5, 50)
y = 60*np.sin(x)
plt.plot(x,y)
plt.show()

# Kurva 2
x = np.linspace(0, 5, 50)
y = np.exp(x)
plt.plot(x,y)
plt.show()
```





1.4 Staplade grafer

Rita flera grafer sidan om varandra.

För att kunna rita så kallade subplots behövs en styrning av var graferna ska placeras. Man tänker i rader och kolumner. Kommandot är `subplot(nrows, ncols, index)`. `nrows` anger antalet rader och `ncols` anger antalet kolumner. Index anger var grafen ska ritas. Index går från vänster till höger och uppifrån och ner.

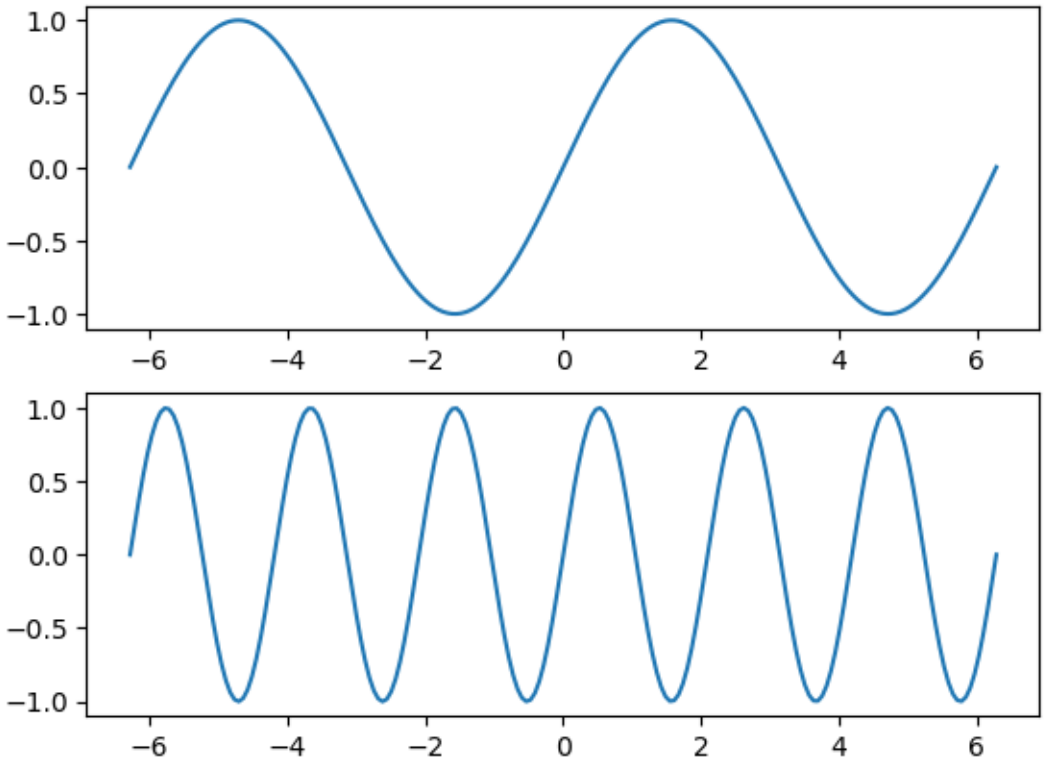
[13]:
```
""" Rita båda funktionerna i två olika grafer samtidigt
subplot(nrows, ncols, index)
"""

import matplotlib.pyplot as plt
import numpy as np

x = np.linspace(-2*np.pi, 2*np.pi, 200)
y = np.sin(x)
plt.subplot(2,1,1) # Två rader, en kolumn, index 1
plt.plot(x,y)

x = np.linspace(-2*np.pi, 2*np.pi, 200)
y = np.sin(3*x)
plt.subplot(2,1,2) # index 2
plt.plot(x,y)
```

```
plt.savefig('plot2.png')
plt.show()
```



Vi gör en 2 gånger 2 och plottar i index 1 och 4. Index 2 och 3 är då tomma.

[4]:
```
"""
Rita båda funktionerna i två olika grafer samtidigt
subplot(nrows, ncols, index)
"""

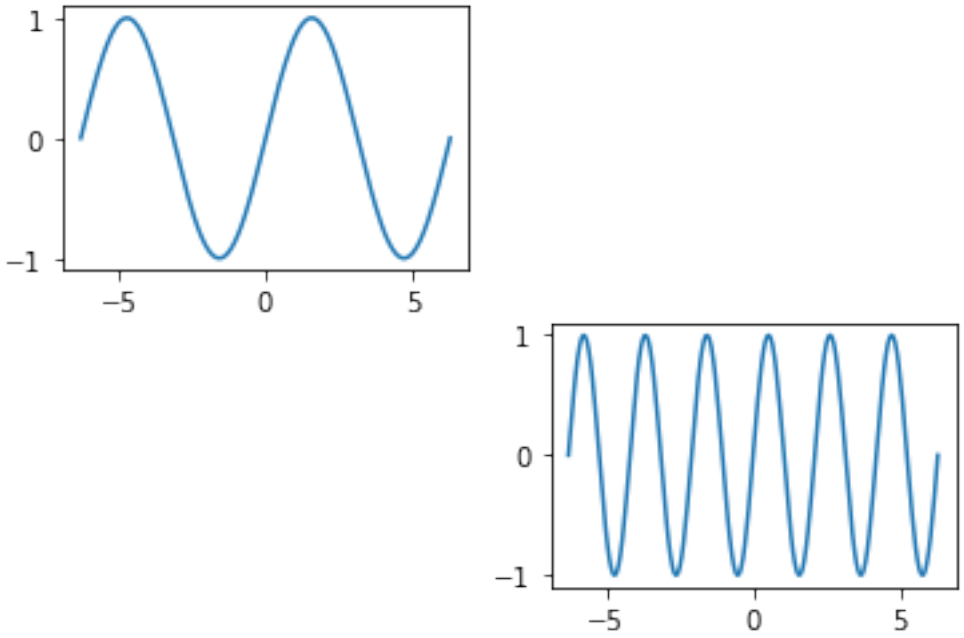
import matplotlib.pyplot as plt
import numpy as np


x = np.linspace(-2*np.pi, 2*np.pi, 200)
y = np.sin(x)
plt.subplot(2,2,1) # Två rader, två kolumneer, index 1
plt.plot(x,y)


x = np.linspace(-2*np.pi, 2*np.pi, 200)
```

```
y = np.sin(3*x)
plt.subplot(2,2,4) # index 4
plt.plot(x,y)


plt.savefig('plot2.png')
plt.show()
```



1.5 Mätdata

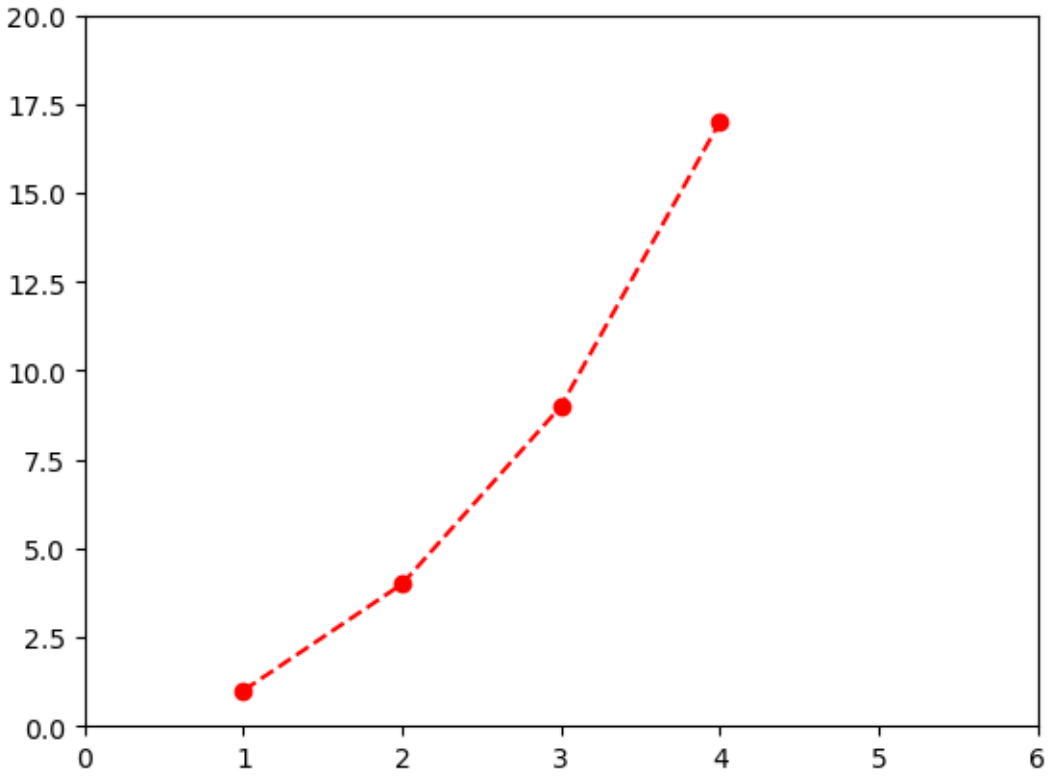
Vi använder mätdata från ett experiment och skriver den i listor. plt.plot tar inte bara array utan även listor. Axlarna bestämmer vi själva denna gång: `plt.axis([0,6,0,20])` anger att x-axeln går från 0 till 6 och y-axeln från 0 till 20. Argumentet är en lista.

[1]:
```
import matplotlib.pyplot as plt
import numpy as np

# Mätdata som skrivits in i listor
x = [1,2,3,4]
y = [1,4,9,17]
plt.axis([0,6,0,20]) # Observera att det är en lista.

plt.plot(x, y, 'ro--')  # röd cirkel, streckad kurva

plt.show()
```



Vill vi inte ha de räta linjerna mellan datapunkterna så heter commandot `plt.scatter(x,y)`. Man kan också utelämna typen av kurva `plt.plot(x, y, 'ro')` så att `--` saknas.

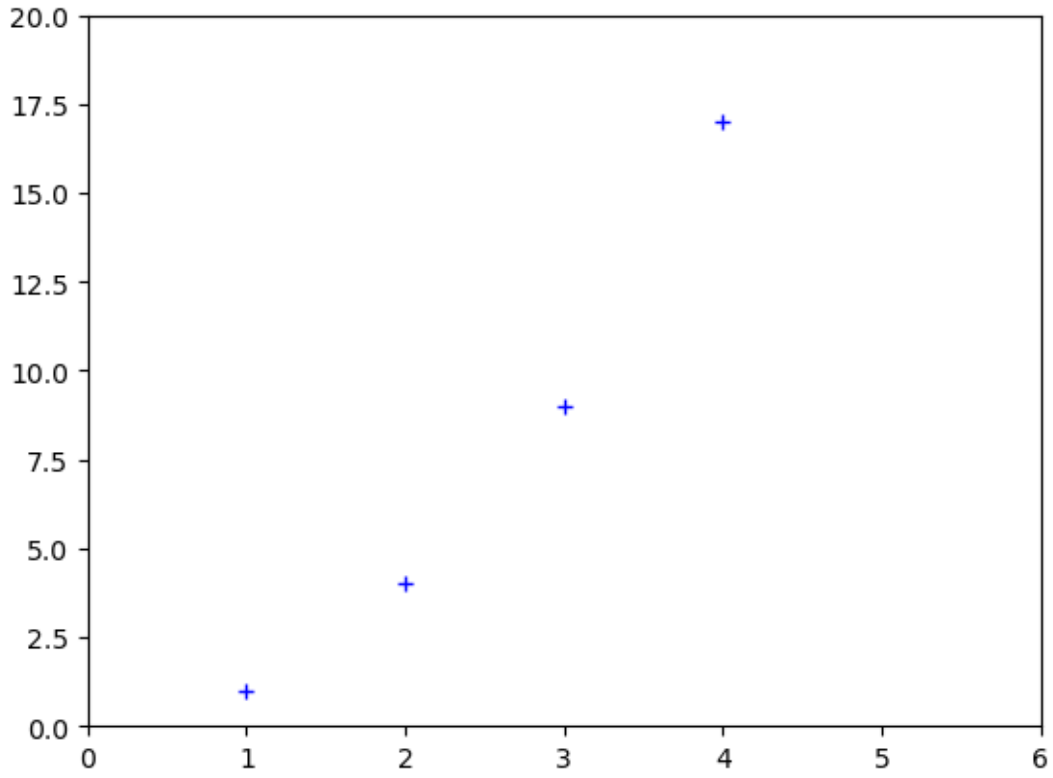
[2]:
```
import matplotlib.pyplot as plt
import numpy as np

# Mätdata som skrivits in i listor
x = [1,2,3,4]
y = [1,4,9,17]
plt.axis([0,6,0,20])

# Nytt kommando! Två alternativ.
# plt.scatter(x, y)

# Eller detta
plt.plot(x, y, 'b+') # blue för blå, markör är + tecken

plt.show()
```



1.6 Teckenförklaring (legend)

Om flera kurvor ritas kan det vara bra att ge information om dem. I detta fall låter vi python bestämma var det finns plats. Kommandona som används är:

`plot(x,y, label='abs(x)')` , `plot(x,y,'k:', label='exp(-x)*sin(x)')` Dvs. vi anger en etikett eller `label()` för att kunna särа på kurvorna.

Teckenförklaringen: `legend(loc='best', fontsize='small')`

[3]:
```
import numpy as np
import matplotlib.pyplot as plt

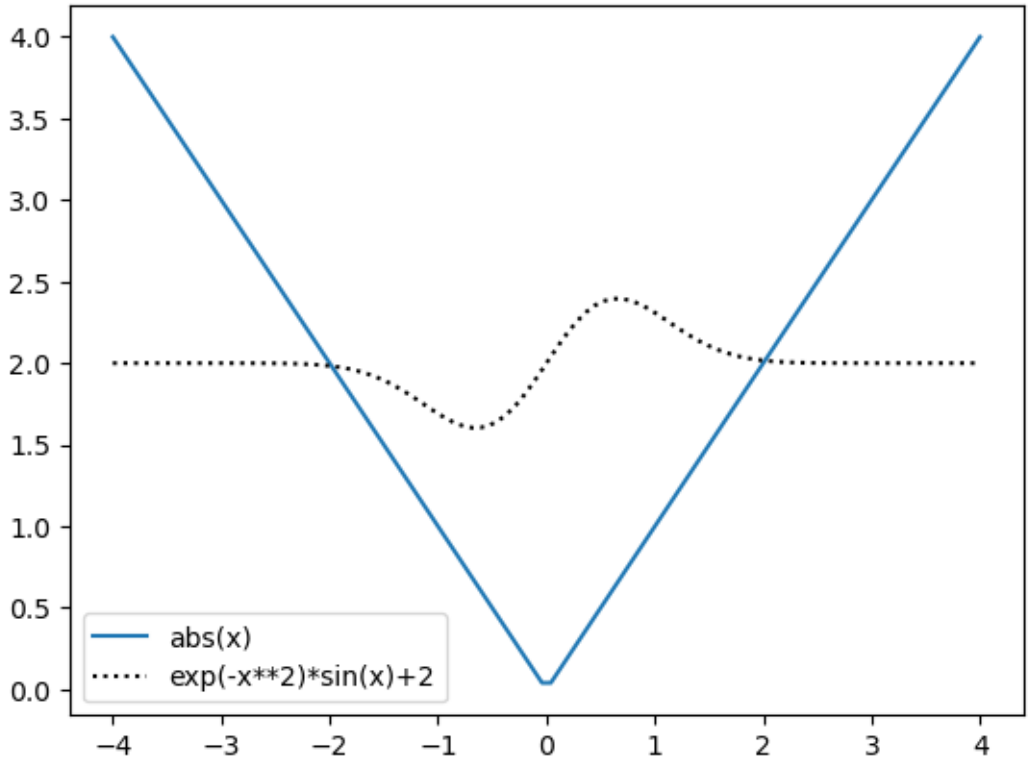
# abs(x)
x = np.linspace(-4, 4, 100)
y = abs(x)
plt.plot(x,y, label='abs(x)')  # Denna kurva får etiketten abs(x)


# exp(-x**2)*sin(x)
x = np.linspace(-4, 4, 100)
y = np.exp(-x**2)*np.sin(x)+2
```

```
plt.plot(x,y,'k:', label='exp(-x**2)*sin(x)+2')  # svart (black) och pricklinje
 ↪(dotted) linje


plt.legend(loc='best', fontsize='medium')

plt.show()
#savefig('plot.png')
```



1.7 Histogram

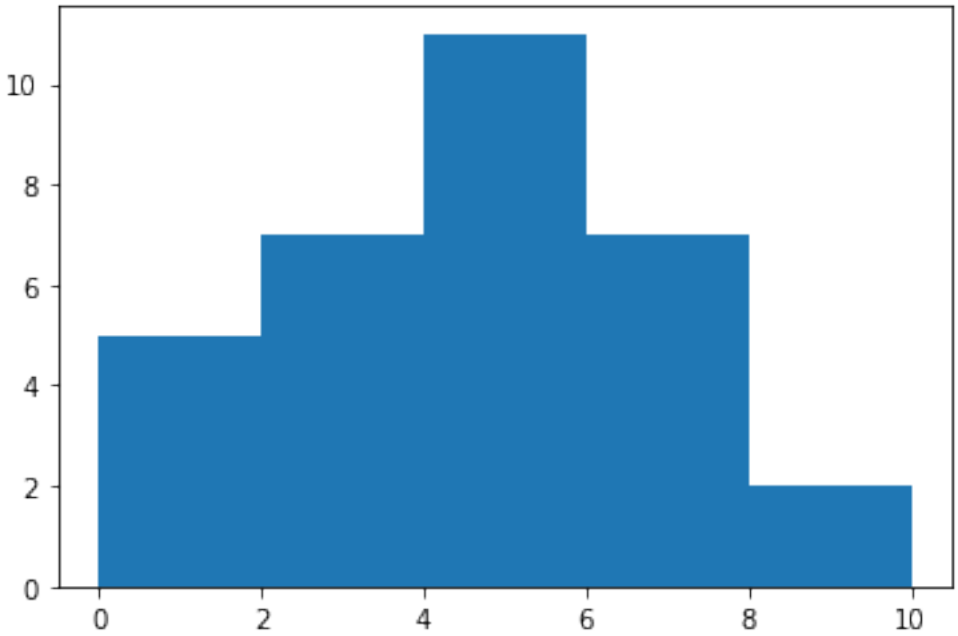
Även histogram kan ritas med matplot.pyplot.

[3]:
```
import matplotlib.pyplot as plt

# Data i lista
x = [1,1,1,1,1,2,2,2,2,2,3,3,
4,4,4,4,5,5,5,5,5,5,5,6,6,6,6,7,7,7,8,8]


# Kommandot för att plotta histogram
plt.hist(x, [0,2,4,6,8,10])
```

```
# De vanliga
plt.savefig("plot.png")
plt.show()
```



Kommentar angående `plt.hist(x, [0,2,4,6,8,10])`

Argumentet består av 2 listor. Datamängden ska finnas i listan x. Listan efter x är listan som anger var gränserna för de olika klasserna (bin, intervall) ska gå. Intervallet from. 0 till 2 är första klassen; alltså inklusive 0 och exklusive 2. För det sista intervallet 8 till 10 gäller dock inklusive för båda gränserna. Vi ser från datamängden att det finns inga 0:or och 5 stycken 1:or vilket ger en stapel med höjden 5 för intervallet 0 till 2; vi ser vidare att det finns 5 st 2:or och 2 st 3:or så 7 blir höjden för intervallet 2 till 4.

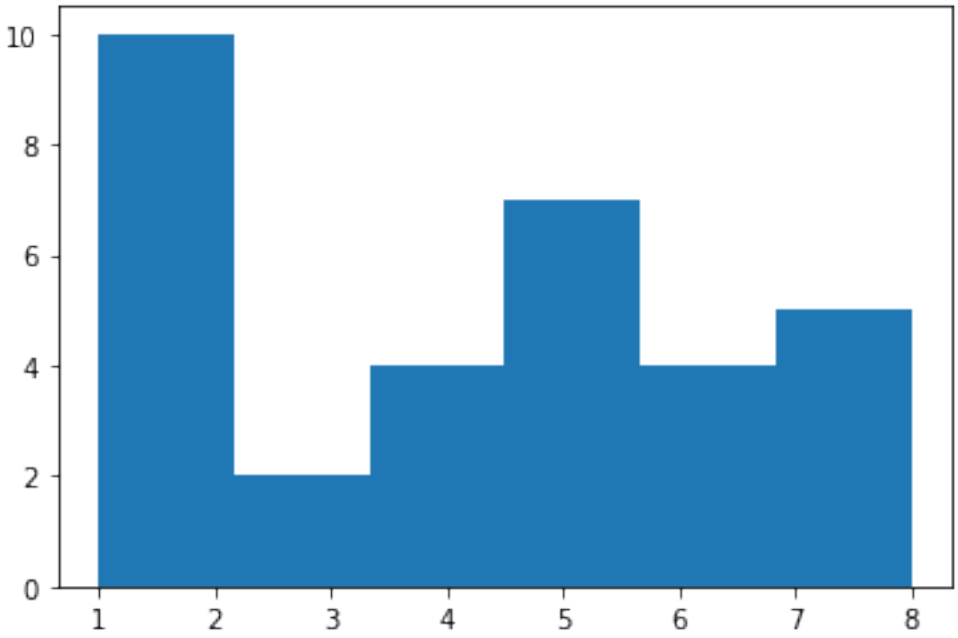
Vi kan också bara ange antalet intervall vi önskar och lämna resten till python som då konstruerar intervall som är lika stora. Anger vi inget intervall så är det förinställt 10 stycken.

[12]:
```
import matplotlib.pyplot as plt

# Data
x = [1,1,1,1,1,2,2,2,2,2,3,3,
4,4,4,4,5,5,5,5,5,5,5,6,6,6,6,7,7,7,8,8]


# Kommandot för att plotta histogram
plt.hist(x, 6)  # 6 intervall
```

```
# De vanliga
plt.savefig("plot.png")
plt.show()
```



`plt.hist(x,6)` är ett anrop precis som när vi skriver en def, den returnerar värden i något som heter en tuple och den fungerar ungefär som en lista; värdena kan adresseras med index som i en lista. Detta innebär att vi kan skriva ut värdena som används för att konstruera grafen numeriskt.

[4]:
```
import matplotlib.pyplot as plt

# Data
x = [1,1,1,1,1,2,2,2,2,2,3,3,
4,4,4,4,5,5,5,5,5,5,5,6,6,6,6,7,7,7,8,8]

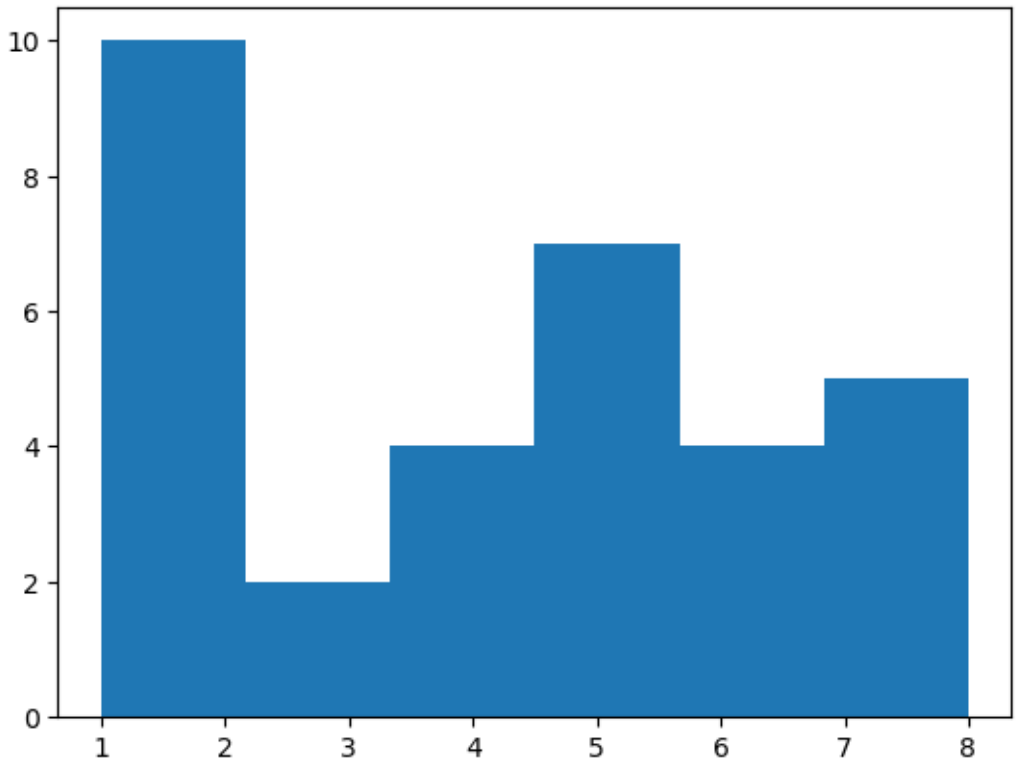

# Kommandot för att plotta histogram, nu med
# return-värdena så vi kan skriva ut dem
n,bins,patches = plt.hist(x, 6)

print("Antalet i varje intervall", n)
print("Binens gränser", bins)
# patches bryr vi oss inte om


# De vanliga
plt.savefig("plot.png")
```

```
plt.show()
```

```
Antalet i varje intervall [10.  2.  4.  7.  4.  5.]
Binens gränser [1.         2.16666667 3.33333333 4.5        5.66666667
6.83333333
 8.        ]
```



[ ]:

1.8 Uppgifter

Uppgift 1

Rita en graf över följande mätdata.

Tid (s)	Läge (m)
0.1	0.00
0.2	0.01
0.3	0.03
0.4	0.06
0.5	0.11
0.6	0.18
0.7	0.26

Tid (s)	Läge (m)
0.8	0.36
0.9	0.45
1	0.54
1.1	0.60
1.2	0.63
1.3	0.59
1.4	0.47
1.5	0.24

Lösningsexempel

Uppgift 2

Rita $y = (x-3)^2$ och $y = (x-5)^2$ och $y = (x+5)^2$ i samma graf. Gränserna i x-led ska vara -10 och 10

Lösningsexempel

Uppgift 3

Rita sin(x) och cos(x) i samma graf. Gränserna ska vara 0 och 2π.

Lösningsexempel

Uppgift 4

Vad är det för skillnad mellan kommandona range(), arange() och linspace()?

Lösningsexempel

Uppgift 5

Konstruera ett histogram ur följande data:

x=[1,1,2,2,1,1,2,2,5,5,6,6,4,4,4,4,5, 5,5,6,6,6,6,7,1,3,2,5,3,7,8,4,4]

med bin/intervall/klasser av storleken 2 med start från 1.

Lösningsexempel

Uppgift 6

I ett experiment erhålls följande data i en lista:

Lx=[1,3,11,24]

Ly=[190,1700,24600,115700]

Det antas utifrån en teori att kurvan som punkterna ungefärligen ligger på ska vara en andragradsfunktion på formen $y = Ax^2$ (ingen x term eller konstant således). Använd datapunkterna för att anpassa funktionen. Kurvan antas dock stämma lite dåligt för första punkten så den kan inte användas för anpassning. Plotta sedan data som + och den anpassade kurvan som en jämn kurva. Använd teckenförklaring (eng. legend).

Lösningsexempel

Uppgift 7

Plotta följande funktioner, välj rimliga axlar.

a) $f_1(x) = |x|$

b) $f_2(x) = e^{-x}\sin(x)$

c) $f_3(x) = \frac{\sin(x)}{x}$

d) $f_4(x) = \sin(1/x)$

Lösningsexempel

Uppgift 8

Rita sin(x). Rita i samma figur Maclaurin polynomen med gradtal 1, 3 och 5, se nedan. Ju högre gradtal man tar med desto bättre ansluter polynomet till sinusfunktionen kring x=0. Gör graferna så att de tydligt kan skiljas åt. Justera axlar, använd färger och olika linjer. Ingen förklaring behöver ges i grafen (ingen 'legend').

$$\sin(x) = x - \frac{x^3}{3!} + \frac{x^5}{5!}$$

Lösningsexempel

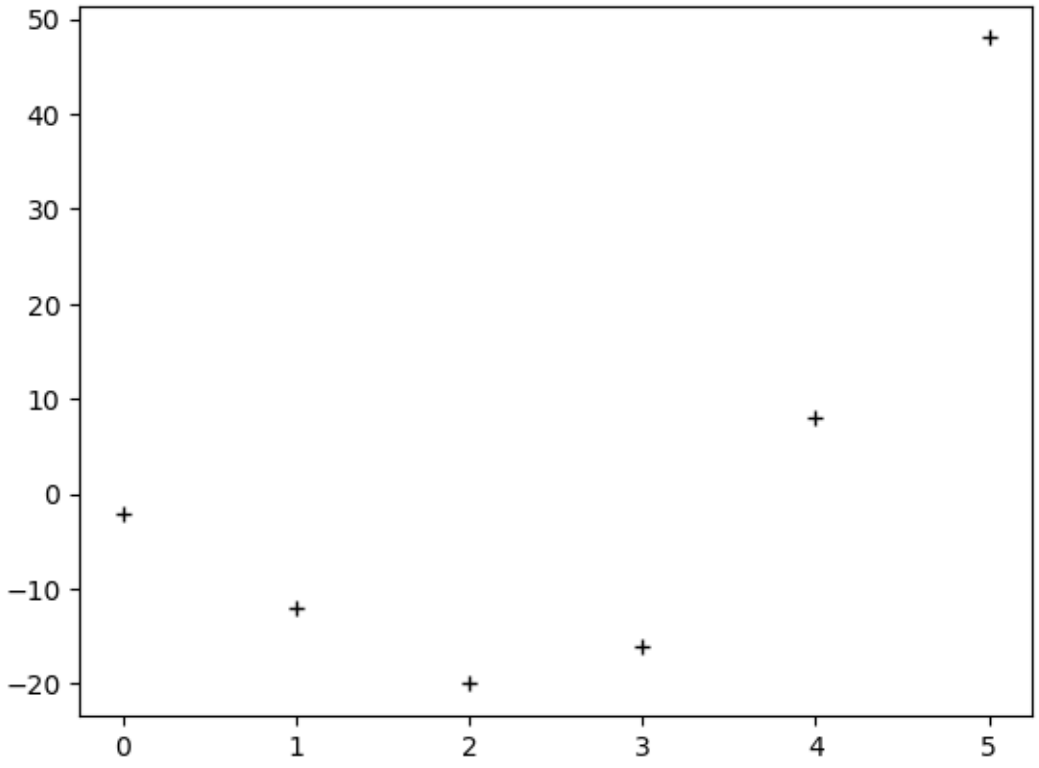
Vi avslutar med att plotta en rekursivt bestämd talföljd som vi haft tidigare. Kopierar in den gamla koden och lägger till kommandon för plottning.

[10]:
```
import matplotlib.pyplot as plt

def rekur(m):
    if m == -1:
        return 4  # a(-1) = 4
    elif m == -2:
        return 5  # a(-2) = 5
    else:
        am = 2*rekur(m-1)-2*rekur(m-2)
        L[m] = am  # Lagrar talen Ny
        return am

print('Hur långt ska beräkningen sträcka sig?\n')
a = int(input('ange ordningsindex '))
L = (a+1)*[0]  # a+1 element Ny
x = range(a+1)  # 0 tom a  Ny
term = rekur(a)
plt.plot(x, L, 'k+')  # Ny
plt.show()  # Ny
```

```
Hur långt ska beräkningen sträcka sig?
```


50
40
30
20
10
0
−10
−20
0
1
2
3
4
5

input2

October 14, 2026

1 Läsa och skriva filer

Behandlar filer av typen .txt (i regel text) respektive .csv (i regel tal).

1.1 open(), close(), with open() as

Ibland är datamängden stor och arbetet med att skriva in den i listor för att rita en graf är stor. Python kan läsa olika filer med data. På samma sätt kan resultatet av beräkningar i python behöva skrivas ut i en fil i stället för att bara flimra förbi på skärmen.

Vi ska arbeta med 2 olika filtyper, de som har typen .txt och de som har typen .csv (anges ibland i ordbehandlingsprogram som 'text csv'). Vi börjar med filtypen .txt. Ingen extra modul behöver laddas för att hantera filer av denna typ.

Processen kan förstås på följande sätt:

1. Filer måste öppnas innan man kan göra något med dem, vi får ett `open()` kommando. Precis som du måste öppna en bok för att läsa i den.
2. Vi måste naturligtvis ange filen som ska öppnas: `open('data.txt')`. Filen ska ha ändelsen txt, en text-fil. Precis som på bibliotek måste du ange vilken bok du ska läsa.
3. Vi måste ange vad vi ska göra: skriva, läsa, lägga till osv.:`open('data.txt', 'r')` för read. Du måste ange om du ska läsa eller skriva. Det är en säkerhetsgrej. Om du bara ska läsa och öppnar för att läsa kommer python hindra dig från att skriva av misstag.

Några exempel på varianter på read och write

- w innebär att filen öppnas för att skrivas i. Om den redan finns töms den och skrivs på nytt. Om filen inte finns skapas den.
- w+ är både läsa och skriva, annars som w.
- a innebär append. Filen öppnas för skrivning, filen töms inte utan man skriver sist i filen. Filen skapas om den inte redan finns.

Kommandot `f = open("datafil1.txt","w")` öppnar filen datafil1.txt och förbereder den för skrivning. Filen får också ett enklare namn, den kallas bara f.

När du är klar med det du ska göra i filen ska man stänga den. Det görs med kommandot `f.close()`.

Det finns ett kommando som gör både open och close, det heter `with open as`. Det används som ett alternativ i exemplet nedan.

[2]:
```
# Öppnar filen för att skriva och ger den ett enklare namn f.
# Vi utgår från att den inte finns.
f = open("datafil1.txt","w")

# Skriver en massa rader till filen som nu har det korta namnet f
for i in range(10):
    text = "Detta är rad " + str(i+1) + "\n"
    f.write(text)  # Varje gång vi skriver blir det en ny rad

# Vi stänger filen när vi är färdiga
f.close()




# För att leka lite med filen så öppnar vi den igen. a står för append
f = open("datafil1.txt", "a")
f.write("Nu var det slut")
f.close()



# Vi skriver ut filen. Det görs genom att öppna den för läsning och sedan läsa.␣
 ↪r read.
# Vi använder denna gång kommandot 'with open() as' som stänger automatiskt.
with open('datafil1.txt', 'r') as f:
    print(f.read())

# filen stängs denna gång automatiskt när indraget upphör.
```

```
Detta är rad 1
Detta är rad 2
Detta är rad 3
Detta är rad 4
Detta är rad 5
Detta är rad 6
Detta är rad 7
Detta är rad 8
Detta är rad 9
Detta är rad 10
Nu var det slut
```

Ändra i föregående kodblock som var

```
# För att leka lite med filen så öppnar vi den igen. a står för append
f = open("datafil1.txt", "a")
f.write("Nu var det slut")
f.close()
```

så att det står “w” i stället för “a”. Då kommer filen att skrivas över och endast “Nu var det slut” kommer att skrivas ut.

Observera således att en fil måste öppnas för ‘read’ för att kunna skrivas ut.

1.2 Läsa hela eller läsa rader

Koden demonstrerar användningen av `read()` respektive `readlines()` för filtypen .txt.

Filen som ska läsas ska finnas. Data i en fil med ändelsen .txt ligger separat. Du kan skapa en sådan fil genom att skriva in rader i en fil med tex. Microsofts Word eller LibreOffice Writer eller enkla texthanterare. Om du skriver i MS Word eller LO Writer så måste filen sparas som .txt!

[7]:
```
# Ska läsa en fil som finns
f = open("datafil2.txt", "r")

# Läs hela filen och skriv ut den
content = f.read()
print("Hela filen direkt\n", content)

f.close()



# Vi öppnar filen igen, läser då automatiskt från början

f = open("datafil2.txt", "r")

# Lägg märke till att filen f1 är en lista med varje rad som ett element i␣
 ↪listan.
# Kommandot är också tydligt readlines().
f1=f.readlines()

print("f1 är av datatypen", type(f1))
print("rad 1:", f1[0])
print("rad 2:", f1[1])
print("rad 3:", f1[2])
print("rad 4:", f1[3])

f.close()
```

```
Hela filen direkt
 Första raden
Sedan kommer andra raden
Och näst sist tredje raden
Slut

f1 är av datatypen <class 'list'>
```

```
rad 1: Första raden

rad 2: Sedan kommer andra raden

rad 3: Och näst sist tredje raden

rad 4: Slut
```

Vi skulle kunna säga att filen är en lista: f1=[“Första raden”, “Sedan kommer andra raden”, “Och näst…]

1.3 Filer med ändelsen .csv

Vi ska nu öppna och läsa en fil med ändelsen (extension) csv (file extension översätts till svenska som filsuffix eller filändelse). csv betyder comma separated values. En fil där data separeras med ett kommatecken (,). SCB benämner dem kommaavgränsad fil. Talen använder decimalpunkt så betydelserna hålls isär. Detta är en sak man måste se upp med då man importerar från olika program. https://fileinfo.com/extension/csv

För att kunna läsa denna typ av filer måste en modul importeras som heter just csv.

Först en bild på den datafil som vi ska läsa. På replit.com skapar du en ny fil och skriver data direkt in. På 3:e raden ligger 2 tal, talet 3 och talet 9. De är separerade med ett kommatecken. Kommatecknet är inte ett decimaltecken i detta sammanhang. Radnummret ingår inte i filen.

[1]:
```
# Vi behöver importera en modul för att läsa .csv. En fil exporterad som 'text␣
 ↪csv'
# från LibreOffice läses.

import matplotlib.pyplot as plt
import numpy as np

import csv

with open('testdata.csv','r') as f:
  data = csv.reader(f)  # Obs reader för csv, ej read!
  print("radutskrift")
  for row in data:  # Behöver ej heta row, men det är suggestivt
        print(row)  # Observera att elementen i listan är strängar
# Automatiskt close i och med vi använt with. Så nästa gång vi använder with␣
 ↪börjar
# vi från början i filen.




# Nu ska vi läsa in dessa som x och y listor.
# Med rader ungefär som ovan läses varje rad som en lista med två 'tal'; de är
```

```
# lagrade som strängar.
# Första elementet i raden är x och det andra y.

x=[]; y=[]  # Två tomma listor att starta med

with open('testdata.csv','r') as f:
  data = csv.reader(f)

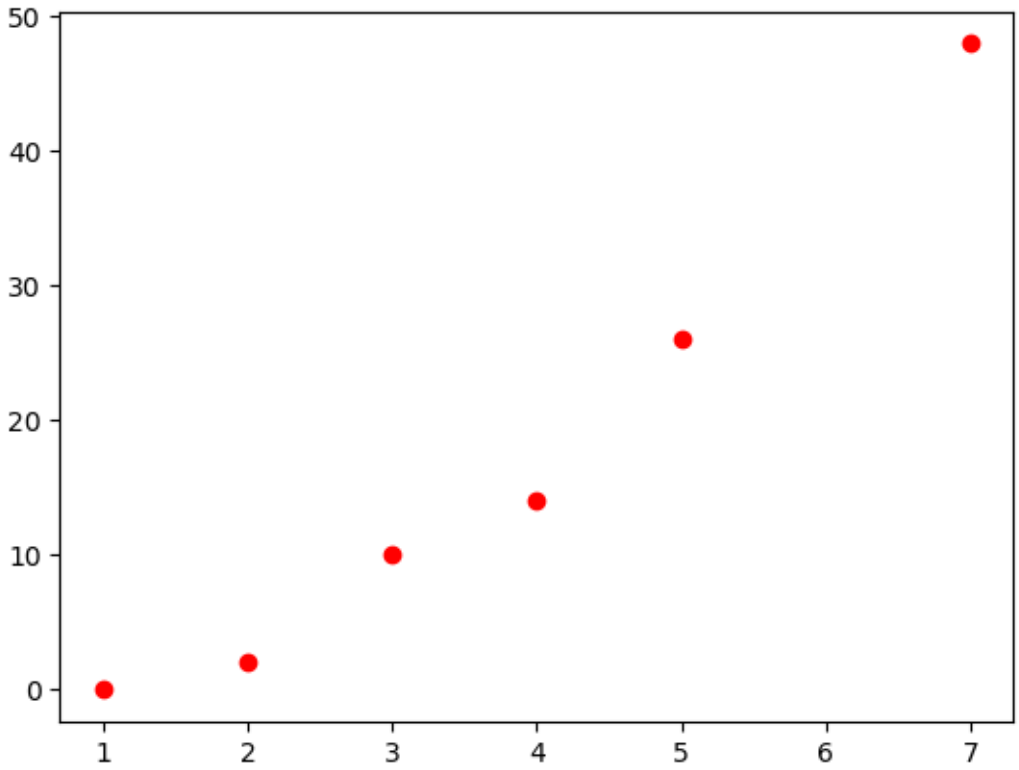
  for row in data:
        x.append(float(row[0]))  # Omvandlar strängarna till float och lägger i
 ↪en annan lista
        y.append(float(row[1]))

print("x och y i listor")
print("x",x)
print("y",y)


plt.plot(x, y, 'ro')
#red circle

plt.savefig('plot.png')
plt.show()
```

```
radutskrift
['1', '0']
['2', '2']
['3', '10']
['4', '14']
['5', '26']
['7', '48']
x och y i listor
x [1.0, 2.0, 3.0, 4.0, 5.0, 7.0]
y [0.0, 2.0, 10.0, 14.0, 26.0, 48.0]
```



Vi tittar i detalj på två centrala delar av koden:

Del 1

```
with open('testdata.csv','r') as f:
  data = csv.reader(f)
  print("radutskrift")
  for row in data:  # Behöver ej heta row, men det är suggestivt
        print(row)
```

Metoden reader stöder iteration, därför kan vi använda det vi fått från reader, data, nästan som en lista, det är dock inte en lista.

`for row in data:` behöver variabeln inte heta `row`, den kan heta vad som helst men det är suggestivt med row.

Del 2

```
with open('testdata.csv','r') as f:
 data = csv.reader(f)
  for row in data:
        x.append(float(row[0]))
        y.append(float(row[1]))
```

Till stor del samma som föregående, row går igenom data rad för rad. Men varje rad består av en sträng, sedan kommatecken och sedan en sträng till. Den första strängen på raden har index 0,

den andra index 1, den tredje index 2 osv. Kommatecknet ingår inte i strängarna.

`row[0]` är den första strängen på den aktuella raden. row itererar sig fram genom raderna.

1.4 Rita med data från kalkylark

Vi kombinerar med en större fil för att rita en graf.

Vi använder att metoden reader för csv filer stöder iteration. Filen testdata2.csv är skapad i LO Calc som är ett program för kalkylark. Se till att decimaltecknet är en punkt, inte kommatecken. File->Save as-> Text csv.

[3]:
```
import csv

import matplotlib.pyplot as plt
import numpy as np

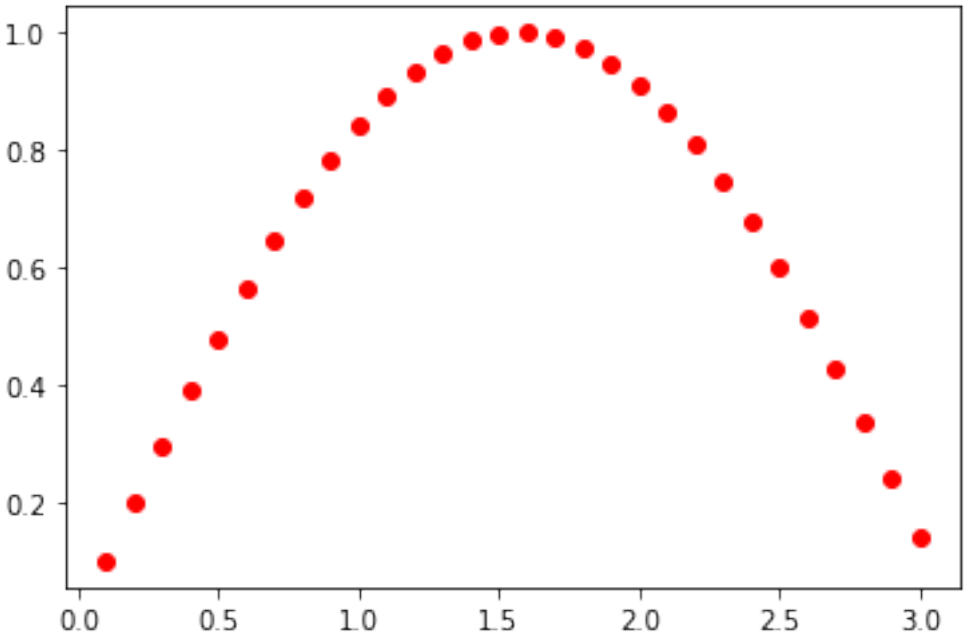

x = []
y = []
with open('testdata2.csv', 'r') as f:
    data = csv.reader(f)
    for row in data:
        x.append(float(row[0]))
        y.append(float(row[1]))

# print("x",x)
# print("y",y)

# plt.axis([0,3,0,1])

plt.plot(x, y, 'ro')
# red circle

plt.savefig('plot.png')
plt.show()
```



1.5 Sammanfattning

Du måste öppna en fil innan du ska använda den, du måste stänga den när du är klar. Du bör också ange vad du ska göra: läsa, skriva. Vi har behandlat 2 filformat: txt och csv. Vi behandlar inte kommersiella format. Alla operativsystem kan spara i txt och csv.

.txt 1. open() 2. close() 3. `with open('datafil1.txt', 'r') as f:` automatisk close(); f är nu kortnamnet för filen du ska arbeta med.
* r, w, w+ 4. f.read när du ska läsa, f.write när du ska skriva 5. f.readlines för att läsa en rad åt gången
6. Rader kan adresseras med index

.csv 1. Kalkylark. Se upp för användningen av decimaltecken; kommatecken eller punkt. 2. Typexempel

```
import csv
with open('testdata.csv','r') as f:
    data = csv.reader(f)
    print("radutskrift")
    for row in data:
        print(row)
```

3. Typexempel inför plot; skapa 2 listor som sedan kan ritas

```
x=[]; y=[] #Två tomma att starta med

with open('testdata.csv','r') as f:
  data = csv.reader(f)
```

```
for row in data:
      x.append(float(row[0]))
      y.append(float(row[1]))
```

1.6 Uppgifter

Uppgift 1

Ladda ner filen, öppna den, läs den och plotta den: uppgift1_fil.csv

Lösningsförslag

Uppgift 2

Hämta en kommaavgränsad fil från statistiska centralbyrån eller ladda ner uppgift2_fil.csv (hämtad från scb 220618) och plotta datamängden. Filen innehåller "Eltillförsel netto, GWh efter månad och produktionsslag". Kommandot `next(data)` gör att en rad hoppas över. Detta kan vara bra då första raden ofta används för rubrik. Koden nedan illustrerar användningen av next().

```
with open('filnam','r') as f:
  data = csv.reader(f)
  next(data)  # ! Hoppa en rad
  for row in data:
        x.append(...)
        y.append(...)
```

Lösningsförslag

[ ]:

moduler2

October 14, 2026

1 Moduler

En modul kan i sin enklaste form bestå av en eller flera def (funktioner) i en fil, med ändelsen .py som vanligt.

1.1 Importera moduler

När man importerar en modul så har den sitt eget *namnrum* (eng. namespace), sin egen uppsättning av namn. Dessa namn kan komma i konflikt med varandra. `sin()` kan betyda olika saker beroende på vilka moduler som är importerade.

Moduler som är vanliga är: math, numpy, scipy, matplotlib, pandas.

Man kan ha `math.sin()` `numpy.sin()` `scipy.sin()`

Tex. tar math bara enkla tal medan numpy tar listor.

För att ta reda på vilka kommandon som finns i modulen scipy skriver man `dir(scipy)`. Man måste först ha gjort en import av modulen.

Om man gör `dir(scipy)` får man inte så mycket information. Man har beslutat att lägga informationen först i underliggande moduler som tex. scipy.stats.

[ ]:
```
import scipy
print(dir(scipy))  # Ger inte så mycket


# Men
import scipy.stats as stats
len(dir(stats))  # Antalet kommandon.

# Om du tar bort # på raden nedan så skrivs alla kommandon ut.
# print(dir(stats))
```

```
['test']
```

[ ]:
```
306
```

Om man inte laddar någon modul så laddas ändå alltid modulen builtins, dvs kommandon inbyggda i pythons grund. Du ska inte ladda den själv det görs automatiskt då python startar.

[12]:
```
print(len(dir(__builtins__)))
print(dir(__builtins__))  # Två understreck på varje sida
```

```
156
['ArithmeticError', 'AssertionError', 'AttributeError', 'BaseException',
'BlockingIOError', 'BrokenPipeError', 'BufferError', 'BytesWarning',
'ChildProcessError', 'ConnectionAbortedError', 'ConnectionError',
'ConnectionRefusedError', 'ConnectionResetError', 'DeprecationWarning',
'EOFError', 'Ellipsis', 'EnvironmentError', 'Exception', 'False',
'FileExistsError', 'FileNotFoundError', 'FloatingPointError', 'FutureWarning',
'GeneratorExit', 'IOError', 'ImportError', 'ImportWarning', 'IndentationError',
'IndexError', 'InterruptedError', 'IsADirectoryError', 'KeyError',
'KeyboardInterrupt', 'LookupError', 'MemoryError', 'ModuleNotFoundError',
'NameError', 'None', 'NotADirectoryError', 'NotImplemented',
'NotImplementedError', 'OSError', 'OverflowError', 'PendingDeprecationWarning',
'PermissionError', 'ProcessLookupError', 'RecursionError', 'ReferenceError',
'ResourceWarning', 'RuntimeError', 'RuntimeWarning', 'StopAsyncIteration',
'StopIteration', 'SyntaxError', 'SyntaxWarning', 'SystemError', 'SystemExit',
'TabError', 'TimeoutError', 'True', 'TypeError', 'UnboundLocalError',
'UnicodeDecodeError', 'UnicodeEncodeError', 'UnicodeError',
'UnicodeTranslateError', 'UnicodeWarning', 'UserWarning', 'ValueError',
'Warning', 'ZeroDivisionError', '__IPYTHON__', '__build_class__', '__debug__',
'__doc__', '__import__', '__loader__', '__name__', '__package__',
'__pybind11_internals_v4_gcc_libstdcpp_cxxabi1011__', '__spec__', 'abs', 'all',
'any', 'ascii', 'bin', 'bool', 'breakpoint', 'bytearray', 'bytes', 'callable',
'chr', 'classmethod', 'compile', 'complex', 'copyright', 'credits', 'delattr',
'dict', 'dir', 'display', 'divmod', 'enumerate', 'eval', 'exec', 'execfile',
'filter', 'float', 'format', 'frozenset', 'get_ipython', 'getattr', 'globals',
'hasattr', 'hash', 'help', 'hex', 'id', 'input', 'int', 'isinstance',
'issubclass', 'iter', 'len', 'license', 'list', 'locals', 'map', 'max',
'memoryview', 'min', 'next', 'object', 'oct', 'open', 'ord', 'pow', 'print',
'property', 'range', 'repr', 'reversed', 'round', 'runfile', 'set', 'setattr',
'slice', 'sorted', 'staticmethod', 'str', 'sum', 'super', 'tuple', 'type',
'vars', 'zip']
```

Man kan kontrollera om ett visst kommando finns i en viss modul. Om den finns svarar python med True.

[2]:
```
print('abs' in dir(__builtins__))
print('open' in dir(__builtins__))  # Kommandot vi använde för att öppna .txt
```

```
True
True
```

[5]:
```
'sin' in dir(__builtins__)
```

[5]: False

[6]:
```
import math

'sin' in dir(math) #Observera inga understreck här
```

[6]:
```
True
```

1.2 Egna moduler

Moduler är pythonprogram med i huvudsak funktioner. Du kan skriva ett pythonprogram själv med funktioner. Sedan kan detta program användas som en modul. Jag kallar modulen/filen nedan i exemplet för *Leifsfkn.py*. Filen innehåller i sin tur importer, det är ok.

Jag har konstruerat 2 egna funktioner som jag kommer använda ofta:

$$\text{sinabscos}(a) = \sin(a)^{|\cos(a)|}$$

och

$$\text{cosabssin}(a) = \cos(a)^{|\sin(a)|}.$$

[3]:
```
import numpy as np

def sinabscos(a):
    return(np.sin(a)**np.abs(np.cos(a)))


def cosabssin(a):
    return(np.cos(a)**np.abs(np.sin(a)))
```

Nu ligger ovanstående fil på servern eller din dator och kan anropas. Filen måste ligga där python söker efter filer. Det är förinställt på en dator men kan ändras. Vi går inte in här på hur detta fungerar på en dator. Arbetar du med replit.com så lägger du bara till en fil i det fönster där du ser att main.py ligger; fältet har rubriken 'Files'. Din fil/modul måste avslutas med .py

Nu skriver vi programmet som anropar min modul med mina hemmasnickrade funktioner.

[8]:
```
import Leifsfkn as Lf

# Observera att det kan bli komplexa tal
print(Lf.sinabscos(2))
print(Lf.cosabssin(1.5))
```

```
0.9612041047552353
0.07120812047572433
```

En bild från repl.it. Vi ser filen main.py. På vänster sida syns också att det finns en fil Leifsfkn.py, den syns på nästa bild.

Lägger du bara filerna under Files så hittar python dem och importerar dem. Du kan nu bygga ditt eget bibliotek med moduler/filer att importera.

1.3 Sammanfattning

- Samla funktions-definitioner i en fil.
- Filens namn ska avslutas med .py
- Lägg den någonstans där python söker.
- Importera filen i ditt huvudprogram; import minafiler.
- Använd filen: minafiler.funktion(…)

1.4 Uppgifter

Uppgift 1

Skriv en modul/fil med 2 funktioner. De är

$$cas(x) = \frac{e^x + e^{-x}}{2}$$

och

$$san(x) = \frac{e^x - e^{-x}}{2}$$

Anropa dem för att rita deras grafer, välj x-värden mellan -2 till 2.

Lösningsförslag

Uppgift 2

Skriv en modul som beräknar approximativt derivatan av en funktion genom beräkning av differenskvot. Till modulen ska skickas intervallet som derivatan ska beräknas i [A,B], antalet punkter för vilket differenskvoten ska beräknas nr och funktionen func. Modulen ska returnera data så func och dess derivata kan plottas

Observera att funktioner kan vara argument när man kallar på en funktion.

```
import numpy as np

def execute (func, argv):
    return func(argv)


execute(np.sin,2)
```

Skriver ut 0.909… som är sinus för 2 radianer.

Lösningsförslag

print2

October 14, 2026

1 Print med formatering

1.1 Metoden .format()

Att skriva ut med formatering kan inledningsvis delas upp i 2 delar. En del som är statisk och en del som är flexibel.

Idén är att ange en sträng (fix) med platshållare för sådant som ska kunna varieras. Ett typiskt exempel är: “Bredden är {} m, längden är {} m och djupet är {} m. Priset: {} kr”. Därefter anger man det som ska in i parenteserna {} med hjälp av en listliknande uppräkning: (2,4,1.5,234)! Parenteserna {} kallas för ersättningsfält (eng. replacement field). Själva strängen kallas ofta för en mall. Talen 2,4,1.5,234 kallas för argument. Dessa argument är i detta fall givna i positioner (man kan ge argumenten som nyckelord också).

Sedan måste man ha ett system för detta. Systemet är att man först anger strängen och sedan det som ska in i parenteserna: sträng.format(). Kommandot får formen:

“Bredden är {} m, längden är {} m och djupet är {} m. Priset: {} kr”.format(2,4,1.5,234)

Vilket ger utskriften: Bredden är 2 m, längden är 4 m och djupet är 1,5 m. Priset: 234 kr.

Vi skriver koden och en utskrift:

[3]:
```
x = "Bredden är {} m, längden är {} m och djupet är {} m. Priset: {} kr".
  ↪format(2,4,1.5,234)

print(x)
```

```
Bredden är 2 m, längden är 4 m och djupet är 1.5 m. Priset: 234 kr
```

Man kan vänta med format till printsatsen:

[7]:
```
x = "Bredden är {} m, längden är {} m och djupet är {} m. Priset: {} kr"

print(x.format(2,4,1.5,234))
```

```
Bredden är 2 m, längden är 4 m och djupet är 1.5 m. Priset: 234 kr
```

Vi ser på lite olika exempel.

[1]:
```
print('Jag har {} och du har {}'.format('en', 'två'))
```

```
Jag har en och du har två
```

[4]:
```
print('Jag såg {} pelikan idag, i går såg jag {}'.format(1, 2))
```

```
Jag såg 1 pelikan idag, i går såg jag 2
```

{} kan dirigera vilket av elementen i listan som ska skrivas ut var. I `.format('en', 'två')` kan `en` adresseras, inte helt oväntat, med index 0.

[1]:
```
print('Jag har {1} och du har {0}'.format('en', 'två'))
```

```
Jag har två och du har en
```

[2]:
```
print("{3} minus {1} är {1}".format("ett","två","tre","fyra"))
```

```
fyra minus två är två
```

Ofta säger man att {} anger var variabler ska skrivas ut.

[2]:
```
x = 11
y = 3
z = x - y

txt = "{0} minus {1} är {2}"

print(txt.format(x,y,z))
```

```
11 minus 3 är 8
```

Se nedan. Följande kanske något omständliga sätt att skriva är vanligt.

[10]:
```
print("{}".format("test"))  # Omständligt
print("test")  # Så skrev vi i början
```

```
test
test
```

1.2 Höger- respektive vänsterjustering.

Vi tittar nu på hur utskriften kan justeras till höger eller vänster på utskriftsraden. Vi kommer att använda det mer omständliga sättet att skriva.

1.2.1 Justering av strängar

Vi börjar med strängar. Det fungerar annorlunda för tal. Kommandona för styrning av utskriften finns i de {} vi använder för att tala om var i en sträng variabler ska skrivas ut.

För att skriva justerat till höger använder vi symbolen > som pekar åt höger.

[9]:
```
print('123456789*')  # För att positionerna ska synas

print('{:_>6}'.format('rest'))  # Understreck _ används för att fylla ut tomma␣
↪positioner
```

```
123456789*
__rest
```

Vi har 6 positioner till 'rest' och i dessa 6 positioner skrivs 'rest' till höger'. 'rest' har t i position 6. Ordets slut är högerjusterat till position 6.

Varför står det kolon i {}? Jo det beror på att man kan ange variabler. I exemplet nedan ska variabeln som heter x skrivas ut. Vi vet dock inte vad den ska vara förrän vi kommer till print-satsen med format och där kan vi ange vilket värde variabeln x ska ha, i detta fall strängen 'rest'. Allmänt så är formen {det_som_ska_skrivas_i_denna_parentes:format_specifikation}. 'ut' tar 2 positioner och sedan tar rest 6 positioner och skrivs högerjusterat; t i resta placeras då i position 2+6. Se nedan.

[1]:
```
print('12345678901234567890')  # För att positionerna ska synas

txt = "ut{x:>6}"
print(txt.format(x='rest'))

# Är samma som (men mindre flexibelt)
txt2 = "ut{:>6}"
print(txt2.format('rest'))

# Ett exempel till...
txt = "ut{x:>6} och gå {y:>3}"
print(txt.format(x='rest', y=5))

print('12345678901234567890')  # För att positionerna ska synas
```

```
12345678901234567890
ut  rest
ut  rest
ut  rest och gå   5
12345678901234567890
```

Så om det står inget till vänster om kolon innanför parentesen `{:>6}` så betyder det att argumenten anges med hjälp av positioner, och de står i `format()`.

[3]:
```
txt = "ut{:>6} och gå {:>3}"
print(txt.format('rest', 5))
```

```
ut  rest och gå   5
```

Strängar justeras till vänster med hjälp av symbolen `<` (som är förinställt).

[15]:
```
print('123456789*')
print('{:_<10}'.format('rest'))
print('{:_<8}'.format('rest'))

print('{:10}'.format('rest')) # < är förinställt, så det behöver egentligen␣
 ↪inte anges
```

```
print('123456789*123')
print('Hej!{:_<8}A'.format('rest'))
# Först 'Hej!' (4 positioner) sedan 8 positioner till 'rest', sedan 'A'
```

```
123456789*
rest______
rest____
rest
123456789*123
Hej!rest____A
```

1.2.2 Justering av tal

Heltal Vanliga sättet att formatera heltal är att skriva till höger (förinställt vänster för strängar men höger för tal). Symbolen `d` talar om för python att vi arbetar med heltal i basen 10 (decimal integer). `d` är också förinställt. Omedelbart efter : kan man också ange en utfyllnadssymbol, här nedan används som exempel _ och * (eng. fill eller padding).

[8]:
```
print("Högerjusterade exempel")
print('123456789*')

print('{:d}'.format(42))  # syns inget intressant
print('{:4d}'.format(42)) # 4 positioner, med förinställt höger >
print('{:_>4d}'.format(42))  # samma som föregående, > förinställt; med _ för␣
 ↪tomma positioner (eng. padding)
print("----\n")
print('{:*>10d}'.format(42))
print('{:10}'.format(42))  # även d kan tas bort eftersom det är förinställt
print("----\n")

# Och så vänsterjusterat
print("Vänsterjusterade exempel")
print('123456789*')

print('{:_<4}'.format(42))  # vänster
print('{:_<4}hej'.format(42))  # vänster 42 tar 4 positioner, därpå följer hej
print('{:_<10d}'.format(42))  # vänster
print('{:_<10d}hej'.format(42))  # vänster 42 tar 10 positioner, därpå följer␣
 ↪hej
```

```
Högerjusterade exempel
123456789*
42
  42
__42
----
```

```
********42
        42
----

Vänsterjusterade exempel
123456789*
42__
42__hej
42________
42________hej
```

Ett exempel på utskrift i annan bas.

x står för hexadecimal.

[18]:
```
print('{:X}'.format(42))  # vänster
```

```
2A
```

$10 \cdot 16^0 + 2 \cdot 16^1$ ty A står för 10 i hexadecimala systemet (B för 11 osv...F för 15)

Flyttal Det är vid formatering av flyttal som det börjar hända saker. f står för fixed point notation, dvs ett givet antal decimaler. Förinställt på 6 decimaler.

[8]:
```
print('Talet pi är cirka {:f}'.format(3.141592653589793))
```

```
Talet pi är cirka 3.141593
```

Se nedan. 6 positioner med 2 efter decimaltecknet. Det är väl denna typ man bör kunna så att man automatiskt styr framförallt antalet decimaler. 6 positioner är just 6 positioner inte 6 siffror. Decimaltecknet upptar en position. 0 framför tvingar fram nollor.

[15]:
```
print('{:>34}'.format(654321))
print('Talet pi med 2 decimaler är {:06.2f}'.format(3.141592653589793))
print()
print('Talet pi med 2 decimaler är {:6.2f}'.format(3.141592653589793))
```

```
                            654321
Talet pi med 2 decimaler är 003.14

Talet pi med 2 decimaler är   3.14
```

Och om vi bara vill ange antalet decimaler

[6]:
```
print('{:.4f}'.format(3.141592653589793))
```

```
3.1416
```

Heltalsexempel med 0 framför.

[18]:
```
print('{:04d}'.format(42))
print('{:4d}'.format(42))  # samma position men inga nollor
```

```
0042
  42
```

Följande direkta användning av listor är också möjlig

[27]:
```
data = [4, 8, 15, 16, 23, 42]
print('{d[4]} {d[5]}'.format(d=data))
```

```
23 42
```

Man behöver se många exempel. Det sägs ofta att den teoretiska definitionen av formateringen är svår att läsa så vi använder endast exempel, vill man verkligen göra något avancerat får man sätta sig ner med det.

[3]:
```
b = input("På vilken tid springer du 100 m? ")
a = int(b)

# Flyttal
print("{:f}".format(a))

#Text också, förinställt på 6 decimaler
print("1 Det var långsamt, jag springer på {:f}".format(a-1))

#Minst 9 positioner, en decimal
print("2 Det var långsamt, jag springer på {:9.1f}".format(a-1))

# 1 decimal
print("3 Det var långsamt, jag springer på {:.1f}".format(a-1))


# 2 decimaler
print("4 Det var långsamt, jag springer på {:.2f}".format(a-1))

print("5 Det var långsamt, jag springer på {:.2f}, förr sprang jag på {:.2f}".
 ↪format(a-1,a-2))
```

```
På vilken tid springer du 100 m? 23
23.000000
1 Det var långsamt, jag springer på 22.000000
2 Det var långsamt, jag springer på      22.0
3 Det var långsamt, jag springer på 22.0
4 Det var långsamt, jag springer på 22.00
5 Det var långsamt, jag springer på 22.00, förr sprang jag på 21.00
```

Fler blandade exempel Sammanfattningsvis hade vi enligt tidigare: {det_som_ska_skrivas_i_denna_parentes:format_specifikation}.

det_som_ska_skrivas_i_denna_parentes kallas tekniskt för fältnamn.
Vi delar också upp format_specifikation i sina delar:
{fältnamn:utfyllnad upplinjering 0 positioner . precision typ}.

- utfyllnad har vi haft som exempel _ och *
- upplinjering har varit < och >. Även ^ finns för mittenjustering.
- 0 står för om nollor ska skrivas ut.
- positioner är antalet positioner som ska användas för de symboler som ska skrivas ut.
- . visar att nu kommer decimaler.
- precision anger antalet decimalet.
- typ står för om det ska vara decimalt heltal (d), exponentialform (e), flyttal (f), grundpotensform (g), procent (%), binär (b) och även sträng (s).

Det finns fler format specifikationer men de behandlar vi inte.

[13]:
```
print("123456789*")
# heltal med minimum bredd på 5 positioner, skriven till höger
print("{:5d}".format(12))

# bredd, 2, som är mindre än talet
print("{:2d}".format(1234))

# 8 positioner, 3 decmaler
print("{:8.3f}".format(12.2346))

# 9 positioner, 3 decimaler
print("{:9.3f}".format(12.2346))

# 10 positioner, 3 decimaler
print("{:10.3f}".format(12.2346))
```

```
123456789*
   12
1234
  12.235
   12.235
    12.235
```

För stora tal har vi

[19]:
```
print("123456789*")
print('{:012.3e}'.format(31415926.53589793))  # Inledande nollor
print('{:12.3e}'.format(31415926.53589793))
print('{:15.3e}'.format(31415926.53589793))
print('{:<15.3e}'.format(31415926.53589793))
print('{:<15.4e}'.format(31415926.53589793))
```

```
123456789*
0003.142e+07
   3.142e+07
      3.142e+07
3.142e+07
3.1416e+07
```

Går också bra med E.

Observera att det verkligen rör sig om positioner, inte siffror. Kommatecken, + och e tar var och en 1 position.

1.3 Avrundning

I detta sammanhang tar vi också upp avrundning. Avrundning fungerar som i matematiken med det vanliga undantaget att tal i bas 10 som inte kan representeras exakt i bas 2 kan ge överraskningar. Principiellt avrundas mittemellan tal till jämn siffra 2,35 blir 2,4; 2,45 blir 2,4. Men man måste se upp eftersom talens representation i bas 2 är det som gäller. Gör man statistiska undersökningar kan det behövas att man slumpar mittemellan avrundningen.

Kommandot är `round(x, n)` där x anger talet som ska avrundas och n anger antalet decimaler. n är villkorligt och kan utelämnas och är förinställt på 0. Formellt skrivs det som `round(x[,n])`

[3]:
```
print(round(2.346))  # Förinställt 0 antal decimaler
```

```
2
```

[4]:
```
print(round(2.346,2))  # Som vi förväntar oss
```

```
2.35
```

[3]:
```
print(round(2.345,2))  # Avrundas upp, men principen säger 2.34
print("{:.53f}".format(2.345))  # Förklaringen: talet är lagrat binärt och är␣
 ↪då större än 2,345

print(round(2.335,2))  # Avrundas ner, principen säger 2.34
print("{:.53f}".format(2.335))  # Förklaringen
```

```
2.35
2.34500000000000019539925233402755111455917358398437500
2.33
2.33499999999999996447286321199499070644378662109375000
2.33499999999999996447286321199499070644378662109375000
```

[15]:
```
print(round(3.875,2))  # Principen säger upp, talet är exakt representerbart i␣
 ↪basen 2
print(format(3.875, '.53f'))
```

```
3.88
3.87500000000000000000000000000000000000000000000000000
```

1.4 Sammanfattning

Kommandona reserverar positioner så att inga andra utskrifter kan hamna i de positioner som reserverats.

Kommandostrukturer att ta med sig (typexempel):

1. print("Bredden är {} m, längden är {} m och djupet är {} m. Priset: {} kr".format(2,4,1.5,234)) Ange en sträng för utskriften med {} som platshållare. Innehållet står i .format().

2. Text. Automatiskt vänsterjusterat: <. För högerjusterat skriv: print(‘{:>6}’.format(‘rest’))
3. Tal. Förinställt höger >.

   3.1 Heltal: print(‘{:>4d}’.format(42))

   3.2 Flyttal: print(“{:9.3f}”.format(12.2346))
4. Avrundning: print(round(2.346,2))

1.5 Uppgifter

Uppgift 1

Skriv ett program som läser in heltalskoefficienter för en andragradsekvation och sedan skriver ut ekvationen snyggt tex. 3x^2+4x-2=0

Lösningsförslag

Uppgift 2

Skriv ett program som skriver ut följande tabell.

```
1234567890ABCDE
Oslo         223
Stockholm    145
Malmö        452
Ystad        553
Luleå        112
Östersund    612
```

Tabellen ska ha 10 positioner för orten (1,2…0) vänsterjusterat och 5 positioner för talen högerjusterat (A…E).

Lösningsförslag

namespace2

October 14, 2026

1 Namespace/Namnrum/Räckvidd

1.1 Globala och lokala variabler

Först en övergripande skiss på hur det är tänkt att fungera i stora drag.

Den stora ramen är programmet i sin helhet. Inom denna ram är *globala variabler* tillgängliga. Globala variabler anges *utanför* definitionerna av funktioner.

Lokala variabler finns inuti definitionerna av funktionerna. Dessa variabler är *inte* tillgängliga utanför den enskilda def.

Se nedan. Variabler i def är inte tillgängliga utanför, i detta fall variabeln b. Att anropa en lokal variabel globalt ger fel.

[1]:
```
def f1(a):
    b = 11  # Här införs b: lokal variabel
    print("b i def = ",b)  # lokal var. kan skrivas ut inuti def
    return(b)

print("f1(2) =", f1(2))
print("b=", b)  # Variabeln b är inte tillgänglig här! Den är lokal, endast i␣
 ↪def. Felmeddelande
```

```
b i def =  11
f1(2) = 11
```

```
---------------------------------------------------------------------------
NameError                                 Traceback (most recent call last)
Cell In[1], line 7
      4     return(b)
      6 print("f1(2) =", f1(2))
----> 7 print("b=", b)

NameError: name 'b' is not defined
```

Koden i ovanstående cell kan representeras i den inledande bilden:

I nedanstående kod har vi nu b som global variabel, den är då tillgänglig överallt även inne i en def.

[1]:
```
def f1(a):
    print("b i def = ",b)  # b är global, tillgänglig även här
    return(b)

b = 5  # Global variabel
print("f1(2) =", f1(2))
print("b=", b)  # Variabeln b är global, tillgänglig här!
```

```
b i def =  5
f1(2) = 5
b= 5
```

Om vi nu anger b som global och sedan har ett b i en def, så är det 2 olika b. Om vi ändrar b i en def så ändras den inte globalt.

[2]:
```
def f1(a):
    b = 7  # Ett lokalt b
    print("b3=", b)
    c = 11
    return (c)


b = 5  # Global variabel
print("f1(2)", f1(2))  # Anrop till def så att satserna där körs, bla. då b = 7
print("b2=", b)  # Denna blev inte ändrad i def
```

```
b3= 7
f1(2) 11
b2= 5
```

Några blandade exempel. Följ variablernas ändringar. När är de globala, när är de lokala?

[3]:
```
def funktion(a):
  # automatiskt ett lokalt a från anropet; python protesterar ej
  print('a först', a)
  a = 6  # ändrar a
  print('a inne i funktionen', a)
  return a

a = 2
s = funktion(3)
print('s', s)
print('a', a)
```

```
a först 3
a inne i funktionen 6
s 6
a 2
```

- Rad 8: a tilldelas värdet 2, en global variabel

- Rad 9: Funktionen anropas, argumentet 3 skickas, funktionen har parametern a som får värdet 3. Vi går till rad 1.
- Rad 3: Vi skriver ut värdet för a inuti funktionen, en lokal variabel. Detta a har värdet 3. Det globala har fortfarande värdet 2.
- Rad 4: Lokala variabeln a ändras till 6. Den globala a ändras inte.
- Rad 5: Vi skriver ut den lokala som nu är ändrad till 6 på rad 4.
- Rad 6: Vi skickar det lokala värdet på a till anropet.
- Rad 9: Anropet tar emot return och ger det till s.
- Rad 10: Skriver ut s, som ju är från return, ett lokalt värde har skickats ut, s=6.
- Rad 11: Lokalt var a=6, men globalt har a alltid varit 2 så print av global ger 2.

Så här gör man inte (se nedan):

[4]:
```
# Bara a, inte god programmeringssed
def funktion(a):
  print('a 1',a)
  a = 12
  return a

a = 11
a = funktion(a)
print('a 2',a)
```

```
a 1 11
a 2 12
```

- Rad 7: a global är 11.
- Rad 8: a global kopieras till a lokal, som finns i def funktion.
- Rad 3: Vi ser att a lokal är 11, en kopia av globala a.
- Rad 4: Vi ändrar a, endast den lokala!
- Rad 5: Den lokala a är ändrad, vi skickar ut den med return.
- Rad 8: Den globala a ändras först här; a-global=funktion(a-global)
- Rad 9: Vi skriver ut global a.

Ännu en att öva på. Håll reda på alla variablernas värden. Skriv gärna a-lokal och a-global. Börja på rad 7. Skriv en lista över vad som skrivs ut. Tryck sedan på Run.

[5]:
```
def funktion(a):
  print('ai', a)
  a = a + 1  # inre, ökar till 12
  return a


a = 11  # yttre
b = funktion(a)
print('ay', a)  # yttre
print('b', b)  # en kopia av inre
```

```
ai 11
ay 11
```

```
b 12
```

1.2 Loopar

Python klarar sig ibland för den är smart konstruerad. Den blandar inte ihop saker fastän de heter samma sak. Men du bör inte skriva så här med i i både den inre och yttre loopen.

[6]:
```
for i in [1,2,3]:
  print('första', i)
  for i in [1,2,3]:
    print('  andra', i)
```

```
första 1
  andra 1
  andra 2
  andra 3
första 2
  andra 1
  andra 2
  andra 3
första 3
  andra 1
  andra 2
  andra 3
```

Skriv så här i stället

[7]:
```
for i in [1,2,3]:
  print('i',i)
  for k in [1,2,3]:
    print(' k',k)
```

```
i 1
 k 1
 k 2
 k 3
i 2
 k 1
 k 2
 k 3
i 3
 k 1
 k 2
 k 3
```

Även om loopar inuti loopar hanteras av python så kan man inte ha samma variabel utanför en loop som innanför.

[8]:
```
i=7  # Vi sätter ett i här
print(i, "\n")
```

```
for i in range(12):  # Vi använder i i en loop, ok. i går från 0 till 11.
    print(i)

print()
print(i, i**3)  # Detta i kommer att vara loopens sista värde på i, dvs. 11 och inte 7
```

```
7

0
1
2
3
4
5
6
7
8
9
10
11

11 1331
```

1.3 Uppgifter

Uppgift 1

Vad ger följande kod för utskrift?

```
def g(a):
    b = 7
    print("b1 =", b)
    return(b)

def h(a):
    b=8
    print("b2 =", b)
    return(b)

b = 5
print("g(0)", g(0))
print("h(0)", h(0))
print("b = ", b)
```

Lösningsexempel